

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐIỆN TỬ I



BÁO CÁO THỰC TẬP TỐT NGHIỆP

Giảng viên : TS. NGUYỄN NGỌC MINH

Học viên : NGUYỄN QUANG BIÊN

Mã học viên : M22CHDT002

Lớp : M22CQDT01-B

Hà Nội, tháng 12 năm 2023

LỜI CẢM ƠN

Qua hơn một năm học tập và rèn luyện tại Học viện Công nghệ Bưu chính Viễn thông, được sự chỉ bảo dạy dỗ đầy tâm huyết của các quý thầy cô. Em đã được truyền đạt rất nhiều kiến thức về lý thuyết cũng như được thực hành trong suốt quãng thời gian học tập ở học viện. Đồng thời, trong thời gian thực tập tại phòng LAB Rạng Đông, khoa Kỹ thuật Điện tử 1, Học viện công nghệ bưu chính viễn thông cũng đã giúp em có được các kinh nghiệm thực tế về công việc, được học hỏi kinh nghiệm, cách xử lý các tình huống xảy ra bất ngờ... Qua đó, em đã hoàn thành kỳ thực tập 7 tuần của mình.

Lời đầu tiên em xin chân thành cảm ơn TS. Nguyễn Ngọc Minh cùng toàn thể các thầy cô giáo của khoa Kỹ thuật Điện tử 1– Học viện công nghệ bưu chính viễn thông đã giúp đỡ em trong toàn bộ quá trình thực tập. Đối với em, đây là một hành trình khó khăn và đầy thử thách, cả trong thời gian học tập cũng như nghiên cứu. Nhưng nhờ sự giúp đỡ và tận tình chỉ dạy của các thầy cô trong quá trình học Thạc sĩ tại trường, em đã có thêm những kiến thức, được tạo nền tảng, dạy cách tư duy, định hướng để em có thể hoàn thành quá trình học tập và thực hiện kỳ thực tập này.

Sau cùng, em xin cảm ơn các anh và các bạn học viên của lớp M22CQDT01-B đã luôn sát cánh và đồng hành cùng em trong quá trình học tập tại trường và hoàn thành báo cáo thực tập tốt nghiệp.

Trong quá trình làm báo cáo thực tập không tránh khỏi những thiếu sót về nội dung, cách diễn giải vấn đề cũng như các lỗi về trình bày. Vì vậy, em rất mong nhận được sự đóng góp ý kiến của quý thầy cô trong Ban lãnh đạo, các anh chị trong công ty để báo cáo đạt được kết quả tốt hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	1
DANH MỤC CÁC BẢNG BIỂU VÀ HÌNH ẢNH	4
PHẦN 1: THỰC TẬP TẠI LAB RẠNG ĐÔNG	7
1.1. GIỚI THIỆU VỀ PHÒNG LAB RẠNG ĐÔNG	7
1.1.1. Một số thông tin về phòng Lab Viễn Thông	7
1.1.2. Các sản phẩm và thiết bị	7
1.2. NỘI DUNG THỰC TẬP TẠI PHÒNG LAB RẠNG ĐÔNG	7
1.2.1. Khái quát về mô hình Edge Computing và một số bài toán ứng dụng	7
1.2.2. Nghiên cứu một số thuật toán phát hiện bất thường cảm biến tại biên	9
1.2.3. Tìm hiểu về phương pháp học máy SVM (Support Vector Machines) và các kỹ thuật học sâu (deep learning)	12
1.2.4. Nghiên cứu quy trình, phương pháp triển khai tinyML trên các thiết bị nhúng ..	20
PHẦN 2: THỰC TẬP CHUYÊN SÂU TẠI LAB RẠNG ĐÔNG	26
2.1. Tổng quan về hệ điều hành Linux	26
2.1.1. Kiến trúc hệ điều hành Linux	26
2.1.2. Linux Kernel Modules	27
2.1.3. Linux driver	29
2.2. Tạo hệ điều hành linux sử dụng nền tảng mã nguồn mở Openwrt	32
2.2.1. Tổng quan về OpenWRT	32
2.2.2. Xây dựng hệ điều hành	33
2.2.3. Cấu hình SSH cho AI7688H	38
2.2.4. Cài Zerotier cho AI7688H	41
2.2.5. Cross Compiler	42
2.2.6. Thêm Package	45

2.2.7. Cấu hình Device Tree	50
2.3. Thử nghiệm.....	51
KẾT LUẬN	59
TÀI LIỆU THAM KHẢO	60

DANH MỤC CÁC BẢNG BIỂU VÀ HÌNH ẢNH

Hình 1. 1: Sơ đồ mô hình kiến trúc của Edge Computing.	8
Hình 1. 2: SVM trong không gian hai chiều và ba chiều.....	12
Hình 1. 3: Tham số C có giá trị nhỏ.....	14
Hình 1. 4: Tham số C có giá trị lớn.....	14
Hình 1. 5: Giá trị gamma lớn	14
Hình 1. 6: Giá trị gamma nhỏ	15
Hình 1. 7: SVM có margin tốt khoảng cách lớn và cân bằng giữa siêu phẳng và các điểm dữ liệu của 2 lớp.....	15
Hình 1. 8: SVM có margin tồi: khoảng cách không cân bằng & nghiêng hẳn về 1 phía	15
Hình 1. 9: Quá trình xây dựng mô hình CNN.....	16
Hình 1. 10: Mô tả hoạt động của thuật toán RNN.	17
Hình 1. 11: Mô hình hoạt động của mạng GAN.....	18
Hình 1. 12: Mô hình học tăng cường sâu.....	19
Hình 1. 13: Sự kết hợp của Machine Learning và Embedded System.	21
Hình 1. 14: Quy trình triển khai TinyML trên các thiết bị nhúng.	23
Hình 1. 15: Các công cụ hỗ trợ tạo code tự động.	25
Hình 2. 1: Sơ đồ khối của một hệ thống máy tính	27
Hình 2. 2: Sơ đồ khối của một hệ thống máy tính.	30
Hình 2. 3: Mối liên hệ giữa Driver, bộ điều khiển và thiết bị dưới góc độ thực thi.	30
Hình 2. 4: Các thành phần của Bus Driver và Device Driver.....	32
Hình 2. 5: Kéo gói Openwrt về	34

Hình 2. 6: Chọn gói hỗ trợ giao diện web Luci	35
Hình 2. 7: Chọn gói hỗ trợ giao thức MQTT	35
Hình 2. 8: Chọn gói zerotier hỗ trợ ssh từ xa.....	36
Hình 2. 9: Chọn thư viện hỗ trợ C++	36
Hình 2. 10: Chọn gói jsoncpp hỗ trợ chuỗi json	36
Hình 2. 11: Quá trình build hoàn tất	37
Hình 2. 12: Chọn file image.....	37
Hình 2. 13: Nạp image cho Ai7688 hoàn tất	38
Hình 2. 14: Check IP (mạng cục bộ).....	38
Hình 2. 15: Kết nối wifi cho board Ai7688.	39
Hình 2. 16: Lưu lại truy cập wifi.....	39
Hình 2. 17: Chọn Gateway ports.....	40
Hình 2. 18: Thêm cấu hình ssh	40
Hình 2. 19: Check IP để ssh (cùng mạng).....	41
Hình 2. 20: SSH từ xa thông qua zerotier	42
Hình 2. 21: Tạo thư mục để crosscompile	43
Hình 2. 22: Thư mục chứa staging dir	43
Hình 2. 23: Kiểm tra toolchain.....	44
Hình 2. 24: Tạo file helloworld.c	44
Hình 2. 25: Biên dịch file helloworld.c.....	45
Hình 2. 26: Chạy file helloworld vừa biên dịch xong ở board Ai7688.	45
Hình 2. 27: Tạo thư mục chứa file helloworld.c	45
Hình 2. 28: Tạo file helloworld.c để làm package	46

Hình 2. 29: Tạo file Makefile cho package helloworld	47
Hình 2. 30: Cập nhật lại Package.	49
Hình 2. 31: Chọn gói helloworld trên menuconfig.	50
Hình 2. 32: Chạy thử package helloworld vừa thêm.	50
Hình 2. 33: Gửi 1 topic publish để blinkled trên board AI7688.	57
Hình 2. 34: Board AI7688 nhận được bản tin topic publish.	57
Hình 2. 35: Thiết bị chạy thật.	58

PHẦN 1: THỰC TẬP TẠI LAB RẠNG ĐÔNG

1.1. GIỚI THIỆU VỀ PHÒNG LAB RẠNG ĐÔNG

1.1.1. Một số thông tin về phòng Lab Viễn Thông

- Tên đầy đủ: Phòng lab Rạng Đông, Học viện công nghệ bưu chính viễn thông
- Trụ sở chính: Tầng 4, Tòa A3, km10 Nguyễn Trãi, Hà Đông, Hà Nội

1.1.2. Các sản phẩm và thiết bị

- Hệ thống nhúng
 - Cung cấp các thiết bị phần cứng và phần mềm cho hệ thống nhúng: Các kit lập trình stm32f1, stm32f4 lập trình trên phần mềm keilC5.
 - Máy tính nhúng: Raspberry pi 4, Intel Galileo Gen 2 Development Kit-Single Board.
- Hệ thống IoT
 - Các sản phẩm và dịch vụ liên quan tới IoT, smart home: đèn thông minh, gateway điều khiển trung tâm, thiết bị theo dõi nhiệt độ, công tắc điện tử
- Thiết bị
 - Hệ thống server cho sinh viên thực hành
 - Máy hàn, máy khô
 - Máy đo oxilo, Nguồn chuẩn

1.2. NỘI DUNG THỰC TẬP TẠI PHÒNG LAB RẠNG ĐÔNG

1.2.1. Khái quát về mô hình Edge Computing và một số bài toán ứng dụng

1.2.1.1. Khái quát về Edge Computing

Edge Computing, hay còn được biết đến như Điện toán biên, không chỉ là một khái niệm kỹ thuật mà còn là bước đột phá mang lại sự linh hoạt và hiệu suất tối ưu cho các hệ thống mạng hiện đại. Trái ngược với việc truyền thống, nơi mà dữ liệu phải chấp nhận cuộc hành trình dài đến trung tâm dữ liệu hay đám mây điện toán trước khi được xử lý, Edge Computing đặt quyền quyết định trực tiếp vào tay các "biên" của mạng. Trong một thời đại nơi mà dữ liệu ngày càng trở nên lớn và phức tạp, Edge Computing đưa ra một giải pháp hiệu quả để giảm thiểu gánh nặng truyền dẫn và tăng cường hiệu suất hệ thống. Thay vì đưa mọi thông tin đến trung tâm dữ liệu hay đám mây để xử lý, Edge cho phép dữ liệu được xử lý ngay tại vị trí nó được tạo ra. Khả năng tính toán không chỉ tồn tại tại những trung tâm xử lý dữ liệu lớn mà

còn được phân tán tới các thiết bị người dùng và thiết bị thông minh. Các biên của mạng không chỉ là các điểm xử lý, mà là những thiết bị thu thập dữ liệu hiệu quả. Từ máy tính xách tay thông thường đến các cảm biến nhỏ trong các thiết bị IoT, tất cả đều có thể trở thành một phần của cơ sở hạ tầng Edge Computing.



Hình 1. 1: Sơ đồ mô hình kiến trúc của Edge Computing.

- Lớp Cloud: đây là một nền tảng dữ liệu lớn giúp phân tích các tác vụ phức tạp và tốn nhiều thời gian như Big Data, Machine Learning ...
- Lớp điện toán biên(Edge Computing): lớp điện toán biên sẽ nằm ngay cạnh hay gần các thiết bị IoT để kết nối và xử lý các dữ liệu cục bộ của hàng tỷ thiết bị IoT. Trong lớp điện toán biên có 3 lớp nhỏ là lớp trung tâm, lớp giữa và lớp ngoài
- Lớp các thiết bị IoT: bao gồm các cảm biến (sensors), các thiết bị đo đạc, điều khiển (controller) ...

Mỗi cảm biến IoT tạo ra hàng tấn dữ liệu mỗi giây. Trong trường hợp điện toán đám mây, dữ liệu ngay lập tức được chuyển đến cơ sở dữ liệu đám mây thống nhất, trung tâm nơi nó được xử lý và lưu trữ. Nếu có bất kỳ hành động nào được yêu cầu, máy chủ trung tâm sẽ gửi phản hồi của nó trở lại thiết bị khi nhận và phân tích dữ liệu thu được. Mặc dù toàn bộ quá trình thường mất ít hơn một giây để hoàn thành, có thể có những tình huống khi phản hồi có thể bị trì hoãn hoặc gián đoạn. Điều này có thể xảy ra do trục trặc mạng, kết nối internet yếu hoặc đơn giản là do trung tâm dữ liệu nằm quá xa thiết bị. Bây giờ, trong trường hợp tính toán biên, bạn không cần gửi dữ

liệu thu được từ các cảm biến IoT ở bất cứ đâu. Bản thân thiết bị hoặc nút mạng gần nhất (ví dụ: bộ định tuyến) chịu trách nhiệm xử lý dữ liệu và có thể phản hồi theo cách phù hợp nếu cần hành động. Cảm biến và thiết bị được triển khai từ xa yêu cầu xử lý thời gian thực. Một hệ thống đám mây tập trung thường xử lý khá chậm trong trường hợp này, đặc biệt khi mà sự ra quyết định phải được thực hiện trong micro giây.

1.2.1.2. Edge computing một số ứng dụng

Edge computing được ứng dụng trong rất nhiều các lĩnh vực trong cuộc sống, chẳng hạn:

- Giải pháp bảo mật: bất kỳ hệ thống bảo mật nào cũng có thể đáp ứng các mối đe dọa bảo mật trong vòng vài giây. Đó là lý do tại sao nó có ý nghĩa để sử dụng Edge Computing cho các hệ thống giám sát. Do đó, với xử lý video trên thiết bị, máy ảnh có thể phát hiện chuyển động, xác định người xâm phạm và cảnh báo ngay lập tức cho người dùng trong trường hợp xâm phạm hoặc hoạt động đáng ngờ. Vì vậy, thay vì chuyển hàng tấn dữ liệu thô đến các máy chủ để xử lý, các camera như vậy sẽ tiết kiệm lưu lượng truy cập internet của bạn, giảm băng thông và lưu trữ đám mây trong khi cải thiện tốc độ và độ chính xác của phản hồi.
- Giảm tải dữ liệu: thay vì tất cả các dữ liệu đều được máy chủ ở cloud xử lý thì thực hiện giảm tải dữ liệu bằng cách tính toán một phần dữ liệu tại biên mạng hoặc nhờ các máy chủ rảnh rỗi tính toán giúp.
- Thiết bị chăm sóc sức khỏe: một trường hợp thực tế cho Edge Computing nằm trong lĩnh vực theo dõi sức khỏe và các thiết bị đeo được khác. Khi được sử dụng trong điều trị từ xa để theo dõi các tình trạng mãn tính của bệnh nhân, họ có thể trở thành những người cứu sống thực sự. Ví dụ, máy đo nhịp tim có khả năng phân tích dữ liệu sức khỏe một cách độc lập, có thể ngay lập tức cung cấp phản ứng cần thiết cho người chăm sóc cảnh báo khi bệnh nhân cần sự giúp đỡ của họ. Phẫu thuật hỗ trợ robot là một case study khác cho Edge Computing trong chăm sóc sức khỏe, đặc biệt là khi mỗi nano giây có thể có nghĩa là sự khác biệt giữa sống và chết. Những robot này cần có khả năng tự phân tích dữ liệu để cung cấp hỗ trợ trong phẫu thuật một cách an toàn, nhanh chóng và chính xác.

1.2.2. Nghiên cứu một số thuật toán phát hiện bất thường cảm biến tại biên

Trong thời đại mạng lưới kết nối đa dạng và phức tạp, nghiên cứu về "Phát hiện bất thường của cảm biến tại biên" đang trở thành một lĩnh vực quan trọng và hấp dẫn

trong ngành công nghiệp IoT và Edge Computing. Cùng với sự bùng nổ của dữ liệu và sự phát triển của Internet of Things (IoT), việc đảm bảo tính ổn định và an toàn của hệ thống trở thành một nhiệm vụ phức tạp, đòi hỏi sự kết hợp độc đáo giữa công nghệ và sự sáng tạo. Nghiên cứu này không chỉ đặt ra câu hỏi về cách chúng ta có thể nâng cao khả năng phát hiện bất thường của các cảm biến tại biên, mà còn tập trung vào việc khám phá những khía cạnh tối ưu hóa trong môi trường đặc biệt của Edge Computing. Tại những "biên" này, nơi mà dữ liệu được sinh ra và tương tác ngay tại chỗ, sự linh hoạt và độ chính xác trở thành chìa khóa để kiểm soát. Các thuật toán như Isolation Forest, One-Class SVM, Autoencoders, và LSTM không chỉ được xem xét về tính hiệu quả mà còn về khả năng ứng dụng trong môi trường Edge Computing đặc biệt. Sau đây là các giới thiệu về các thuật toán trên:

- Thuật Toán Isolation Forest, một thuật toán phát hiện bất thường độc đáo, đưa ra một cách tiếp cận mới để xử lý thách thức của việc nhận diện dữ liệu ngoại lệ trong môi trường ngày nay. Với ý tưởng cơ bản rằng điểm dữ liệu bất thường ít hơn điểm bình thường, Isolation Forest xây dựng các cây quyết định ngắn gọn và cô đọng. Mỗi cây được tạo ra bằng cách chọn ngẫu nhiên một điểm dữ liệu và tách dữ liệu thành hai phần bằng cách sử dụng giá trị ngưỡng ngẫu nhiên. Sự cô đọng của mỗi điểm dữ liệu được đo bằng chiều sâu của nó trong cây, với sự cô đọng cao tương đương với sự nhanh chóng bị tách ra khỏi phần còn lại. Thuật toán này không yêu cầu quá trình chuẩn hóa dữ liệu và có tốc độ xử lý cao, đặc biệt hiệu quả đối với dữ liệu đa dạng và phức tạp. Tuy nhiên, cũng cần lưu ý đến nhược điểm của Isolation Forest. Trong một số tình huống, nó có thể không đạt hiệu suất tốt trên các tập dữ liệu có cấu trúc đặc biệt hoặc khi đối mặt với nhiều dữ liệu nhiễu. Điều này có thể dẫn đến số lượng dự đoán sai lớn và đòi hỏi sự cân nhắc kỹ lưỡng về điều kiện áp dụng để đảm bảo sự tin cậy và hiệu suất cao nhất. Isolation Forest, với sự cân nhắc chặt chẽ và ánh sáng mới về cách chúng ta nhìn nhận về phát hiện bất thường, mang lại một công cụ mạnh mẽ trong thế giới ngày nay, nơi dữ liệu ngày càng trở nên đa dạng và phức tạp.
- Thuật toán One-Class Support Vector Machine (One-Class SVM) là một công cụ mạnh mẽ trong lĩnh vực máy học, đặc biệt được thiết kế để giải quyết vấn đề phát hiện bất thường khi chỉ có dữ liệu từ một lớp duy nhất, tức là chỉ có dữ liệu bình thường mà không có dữ liệu ngoại lệ. Với đặc điểm là sự linh hoạt và khả năng áp dụng trong nhiều tình huống, One-Class SVM hoạt động dựa trên ý tưởng tạo ra một biên quyết định tốt nhất giữa dữ liệu bình thường và không gian ngoại lệ. Quá trình này đưa ra một cách tiếp cận độc đáo để phân loại và đo lường sự "bình thường" trong dữ liệu mà không cần sự hỗ trợ của dữ liệu ngoại lệ. One-Class SVM có khả năng xây dựng một biên siêu phẳng

dựa trên dữ liệu bình thường, với mục tiêu là giữ cho tất cả các điểm dữ liệu thuộc lớp này ở một bên của siêu phẳng. Điều này cho phép thuật toán xác định được phần lớn của không gian không gian dữ liệu bình thường và, nhờ đó, có thể phát hiện được bất kỳ điểm nào nằm ngoài phạm vi dự đoán là ngoại lệ. One-Class SVM không yêu cầu giả định nào về phân phối của dữ liệu bình thường và hoạt động hiệu quả trong các tình huống mà dữ liệu ngoại lệ có thể là hiếm và khó xác định. Điều này làm cho thuật toán trở thành một lựa chọn ưu việt khi ta chỉ có dữ liệu tích cực và mong muốn phát hiện sự bất thường một cách chính xác và linh hoạt.

- Autoencoders là một lớp mô hình máy học không giám sát có khả năng học biểu diễn tối ưu của dữ liệu đầu vào thông qua quá trình tự học. Được thiết kế dựa trên kiến trúc neural network, Autoencoders có khả năng tự mã hóa và giải mã thông tin, giúp chúng ta hiểu biểu diễn nén của dữ liệu mà không cần nhãn. Thuật toán này hoạt động bằng cách chia quá trình học thành hai phần chính: phần mã hóa (encoder) và phần giải mã (decoder). Phần mã hóa giảm chiều số chiều của dữ liệu đầu vào xuống một biểu diễn nén (latent space) có chiều thấp hơn. Phần giải mã sau đó tái tạo dữ liệu từ biểu diễn nén, cố gắng sao cho dữ liệu tái tạo gần giống nhất với dữ liệu ban đầu. Một trong những ưu điểm lớn của Autoencoders là khả năng học được các biểu diễn tối ưu của dữ liệu mà không cần sự giám sát. Điều này có thể hữu ích trong nhiều ứng dụng, bao gồm phân loại, giảm chiều dữ liệu, và đặc biệt là trong phát hiện bất thường (anomaly detection), nơi Autoencoders có thể học biểu diễn của dữ liệu bình thường và từ đó phát hiện các điểm ngoại lệ. Với khả năng học các biểu diễn phức tạp và không tập trung vào những đặc trưng quan trọng của dữ liệu, Autoencoders trở thành một công cụ quan trọng trong thế giới của học máy và trí tuệ nhân tạo, đặc biệt là trong bối cảnh xử lý dữ liệu đa dạng và phức tạp ngày nay.
- Long Short-Term Memory (LSTM) là một trong những kiến trúc mạng neural network đặc biệt được thiết kế để xử lý vấn đề của việc lưu giữ thông tin trong mạng neural qua thời gian dài mà không gặp phải vấn đề evaporation hoặc vanishing gradients như các mô hình truyền thống khác. LSTM được xây dựng dựa trên kiến trúc RNN (Recurrent Neural Network), nhưng nó cải tiến bằng cách sử dụng các "cổng" (gates) để kiểm soát việc truyền thông tin qua các bước thời gian. Nhờ có các cổng này như cổng quên (forget gate), cổng đầu vào (input gate), và cổng đầu ra (output gate), LSTM có khả năng "gạt" hoặc "lọc" thông tin quan trọng cần được lưu trữ và sử dụng. Điều này giúp mô hình đánh giá được độ quan trọng của từng phần tử dữ liệu, giữ được thông tin quan trọng qua nhiều thời điểm và tránh được việc mất mát thông tin quá nhanh.

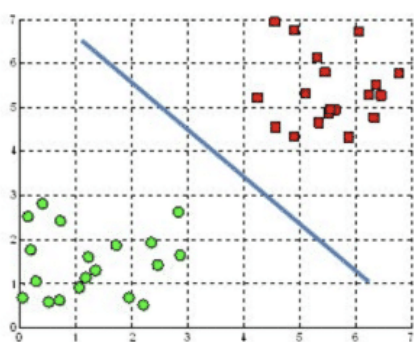
Một trong những ưu điểm chính của LSTM là khả năng xử lý và mô hình hóa các chuỗi dữ liệu dài với sự phức tạp và các mối quan hệ lâu dài giữa các thành phần. Điều này làm cho LSTM trở thành công cụ mạnh mẽ trong việc dự đoán chuỗi thời gian, xử lý ngôn ngữ tự nhiên, và nhiều ứng dụng trong lĩnh vực dự đoán chuỗi, nhận diện ngôn ngữ, và dữ liệu chuỗi có cấu trúc. Với khả năng lưu giữ thông tin qua thời gian mà không bị triệt tiêu, LSTM đã trở thành một công cụ cực kỳ quan trọng trong lĩnh vực trí tuệ nhân tạo và học máy, mở ra nhiều cánh cửa cho việc nghiên cứu và ứng dụng trong các lĩnh vực như ngôn ngữ tự nhiên, dự đoán chuỗi thời gian, và xử lý dữ liệu chuỗi phức tạp.

1.2.3. Tìm hiểu về phương pháp học máy SVM (Support Vector Machines) và các kỹ thuật học sâu (deep learning)

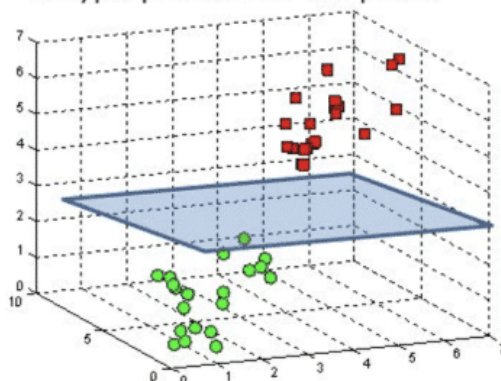
1.2.3.1. Thuật toán Support Vector Machine – SVM

SVM là một thuật toán giám sát, nó có thể sử dụng cho cả việc phân loại hoặc đệ quy. Tuy nhiên nó được sử dụng chủ yếu cho việc phân loại. Trong thuật toán này, chúng ta vẽ đôi thị dữ liệu là các điểm trong n chiều (ở đây n là số lượng các tính năng bạn có) với giá trị của mỗi tính năng sẽ là một phần liên kết. Sau đó chúng ta thực hiện tìm "đường bay" (hyper-plane) phân chia các lớp. Hyper-plane nó chỉ hiểu đơn giản là 1 đường thẳng có thể phân chia các lớp ra thành hai phần riêng biệt. Là một kỹ thuật phân lớp khá phổ biến, SVM thể hiện được nhiều ưu điểm trong số đó có việc tính toán hiệu quả trên các tập dữ liệu lớn.

A hyperplane in $\mathbb{R}^2$ is a line



A hyperplane in $\mathbb{R}^3$ is a plane



Hình 1. 2: SVM trong không gian hai chiều và ba chiều

Ưu điểm:

- Xử lý trên không gian số chiều cao: SVM là một công cụ tính toán hiệu quả trong không gian chiều cao, trong đó đặc biệt áp dụng cho các bài toán phân loại văn bản và phân tích quan điểm nơi chiều có thể cực kỳ lớn
- Tiết kiệm bộ nhớ: Do chỉ có một tập hợp con của các điểm được sử dụng trong quá trình huấn luyện và ra quyết định thực tế cho các điểm dữ liệu mới nên chỉ có những điểm cần thiết mới được lưu trữ trong bộ nhớ khi ra quyết định
- Tính linh hoạt - phân lớp thường là phi tuyến tính. Khả năng áp dụng Kernel mới cho phép linh động giữa các phương pháp tuyến tính và phi tuyến tính từ đó khiến cho hiệu suất phân loại lớn hơn.

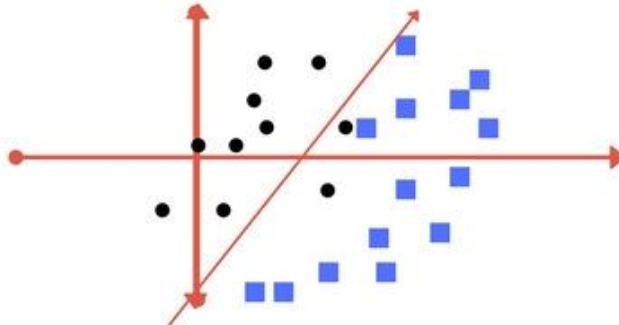
Nhược điểm:

- Bài toán số chiều cao: Trong trường hợp số lượng thuộc tính (p) của tập dữ liệu lớn hơn rất nhiều so với số lượng dữ liệu (n) thì SVM cho kết quả khá tồi
- Chưa thể hiện rõ tính xác suất: Việc phân lớp của SVM chỉ là việc cố gắng tách các đối tượng vào hai lớp được phân tách bởi siêu phẳng SVM. Điều này chưa giải thích được xác suất xuất hiện của một thành viên trong một nhóm là như thế nào. Tuy nhiên hiệu quả của việc phân lớp có thể được xác định dựa vào khái niệm margin từ điểm dữ liệu mới đến siêu phẳng phân lớp mà chúng ta đã bàn luận ở trên.

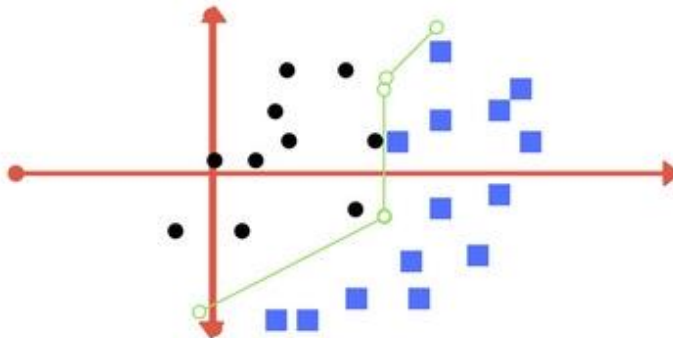
Các tham số chính bao gồm:

- **Kernel** là quá trình xây dựng siêu phẳng phân cách trong SVM được thực hiện qua các phép biến đổi đại số. Với kernel dạng tuyến tính (linear kernel) công thức dùng để dự đoán những điểm dữ liệu mới là: thực hiện tích vô hướng giữa đầu vào (x) với mỗi support vector (x_i) như sau: $f(x) = B(0) + \sum(a_i (x, x_i))$. Các hệ số $B(0)$ và a_i (cho mỗi đầu vào) phải được ước tính từ dữ liệu học. Với kernel dạng đa thức (polynomial kernel) có thể được viết dưới dạng: $K(x, x_i) = 1 + \sum(x, x_i)^d$. Còn với kernel dạng lũy thừa (exponential kernel) có dạng: $K(x, x_i) = \exp(-\gamma \sum((x - x_i)^2))$. Kernel dạng đa thức và dạng lũy thừa tính toán đường phân cách ở những chiều không gian cao hơn và được gọi là kernel trick.
- **Regularization** là Tham số Regularization (được nhắc đến trong thư viện sklearn là tham số C) điều chỉnh việc có nên bỏ qua các điểm dữ liệu bất thường trong quá trình tối ưu mô hình SVM. Nếu tham số này có giá trị lớn, quá trình tối ưu sẽ chọn một siêu phẳng sao cho siêu phẳng này phân cách tất cả các điểm dữ liệu một cách tốt nhất, từ đó khoảng cách giữa siêu phẳng tới các điểm dữ liệu của 2 lớp sẽ có giá trị nhỏ (small-margin). Ngược lại, khi tham số này có giá trị nhỏ, siêu phẳng sẽ được xây dựng sao cho khoảng cách với các điểm dữ liệu của 2 lớp

có giá trị lớn (large-margin), kể cả khi siêu phẳng này sẽ phân loại sai nhiều điểm dữ liệu hơn.

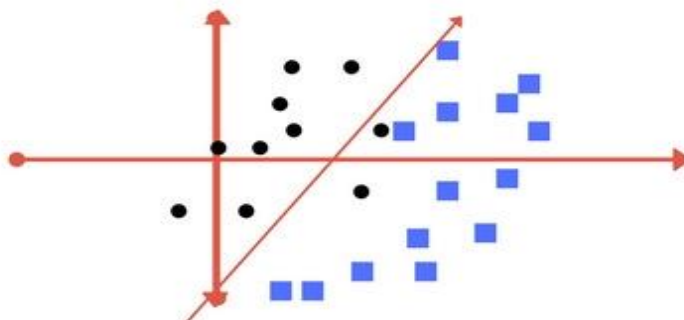


Hình 1. 3: Tham số C có giá trị nhỏ

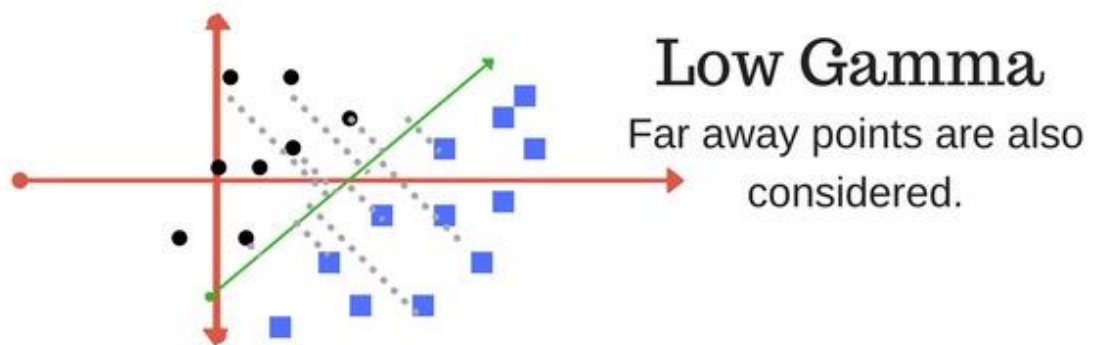


Hình 1. 4: Tham số C có giá trị lớn.

- **Gamma** là Tham số gamma xác định việc sử dụng bao nhiêu điểm dữ liệu cho việc xây dựng siêu phẳng phân cách. Với giá trị gamma nhỏ, các điểm dữ liệu nằm xa đường phân cách sẽ được sử dụng trong việc tính toán đường phân cách. Ngược lại, với giá trị gamma lớn, chỉ những điểm nằm gần đường phân cách mới được sử dụng để tính toán.

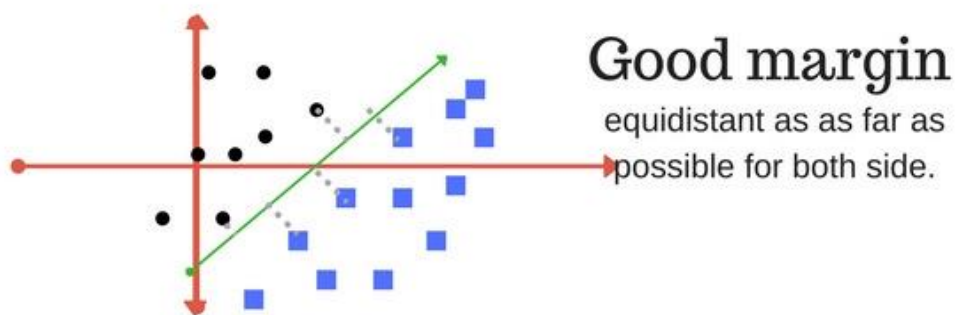


Hình 1. 5: Giá trị gamma lớn

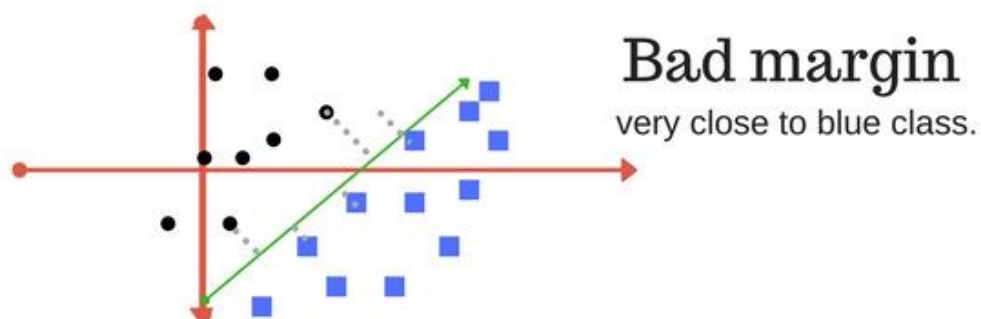


Hình 1. 6: Giá trị gamma nhỏ

- Margin trong SVM là khoảng cách giữa siêu phẳng phân cách với các điểm dữ liệu gần nó nhất. Khoảng cách này đối với các điểm dữ liệu gần nhất của cả 2 lớp càng lớn thì mô hình càng phân loại chính xác.



Hình 1. 7: SVM có margin tốt khoảng cách lớn và cân bằng giữa siêu phẳng và các điểm dữ liệu của 2 lớp.



Hình 1. 8: SVM có margin tồi: khoảng cách không cân bằng & nghiêng hẳn về 1 phía

1.2.3.2. Các kỹ thuật học sâu (deep learning)

1. Mạng nơ-ron cổ điển

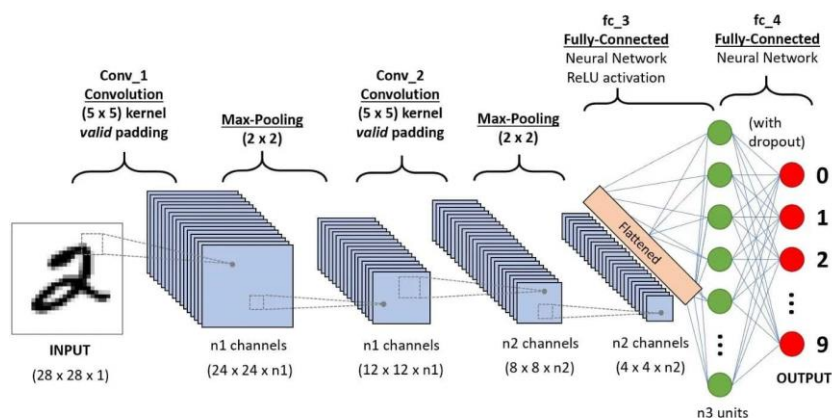
Mạng nơ-ron cổ điển sử dụng kiến trúc mạng kết nối đầy đủ, thông thường xây dựng từ các perceptron đa tầng, nhằm phân loại các dữ liệu. Perceptron là một thuật toán đơn giản dùng để tìm ranh giới siêu phẳng cho bài toán phân lớp nhị phân. Fran Rosenblatt đã thiết kế mạng nơ-ron cổ điển vào năm 1958, và phương pháp này chủ yếu được áp dụng trong các bài toán phân lớp nhị phân. Có ba loại hàm thường được sử dụng trong mô hình này:

- Hàm tuyến tính.
- Hàm phi tuyến, bao gồm sigmoid, tanh và ReLU (Rectified Linear Unit).

Kiến trúc mạng nơ-ron cổ điển đơn giản và phù hợp nhất với các dữ liệu có cấu trúc bảng hoặc các bài toán phân loại và hồi quy với đầu vào là giá trị số thực.

2. Mạng nơ-ron tích chập (CNN)

Convolutional Neural Network (CNN) là một kiến trúc Neural Network tiên tiến, được phát triển để giải quyết các bài toán phức tạp, đặc biệt là trong lĩnh vực xử lý hình ảnh. Tích chập là một khái niệm trong xử lý tín hiệu số, sử dụng phép tích chập với bộ lọc để biến đổi thông tin đầu vào và tạo ra một tín hiệu mới. Quá trình này loại bỏ các đặc trưng không quan trọng và giữ lại những đặc trưng quan trọng và hữu ích nhất. Mô hình CNN bao gồm input layer, output layer và thêm một sampling layer để hạn chế số lượng nơ-ron trong các layer tương ứng.



Hình 1. 9: Quá trình xây dựng mô hình CNN.

3. Mạng nơ-ron hồi quy (RNN)

Trong lĩnh vực xử lý ngôn ngữ tự nhiên, Recurrent Neural Network (RNN) là một thuật toán được đánh giá cao. Trong các mô hình mạng nơ-ron truyền thống, đầu vào và đầu ra được xử lý độc lập với nhau. Tuy nhiên, RNN thực hiện cùng một tác vụ cho tất cả các phần tử trong một chuỗi dữ liệu, và đầu ra phụ thuộc vào cả quá trình tính toán trước đó. Điều này cho phép mạng RNN ghi nhớ thông tin từ các phép tính trước đó.

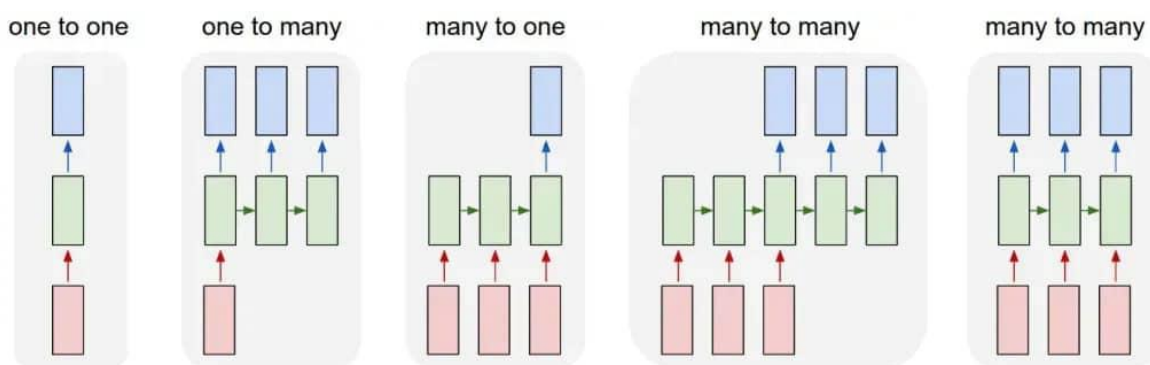
RNN có hai thiết kế chính:

- LSTM (Long Short-Term Memory): Được sử dụng để dự đoán dữ liệu chuỗi thời gian. LSTM có khả năng xóa hoặc thêm thông tin vào bộ nhớ dài hạn (long-term memory) thông qua các cổng (gates) như Input, Output và Forget.
- Gated RNN: Đây cũng là một thiết kế phổ biến trong việc dự đoán dữ liệu chuỗi thời gian. Gated RNN có hai cổng chính là Update và Reset.

RNN có thể được áp dụng vào các dạng bài toán sau:

- One to one: Một đầu vào tương ứng với một đầu ra duy nhất, ví dụ như bài toán phân loại hình ảnh.
- One to many: Một đầu vào tương ứng với nhiều đầu ra chuỗi, phổ biến trong các bài toán đặt chú thích (captioning) cho ảnh.
- Many to one: Nhiều đầu vào nhưng chỉ có một đầu ra, ví dụ như bài toán phân loại cảm xúc.
- Many to many: Nhiều đầu vào và nhiều đầu ra, ví dụ như bài toán phân loại video.

Mạng RNN mang lại khả năng xử lý dữ liệu chuỗi tự nhiên một cách hiệu quả và đã được áp dụng thành công trong nhiều ứng dụng.



Hình 1. 10: Mô tả hoạt động của thuật toán RNN.

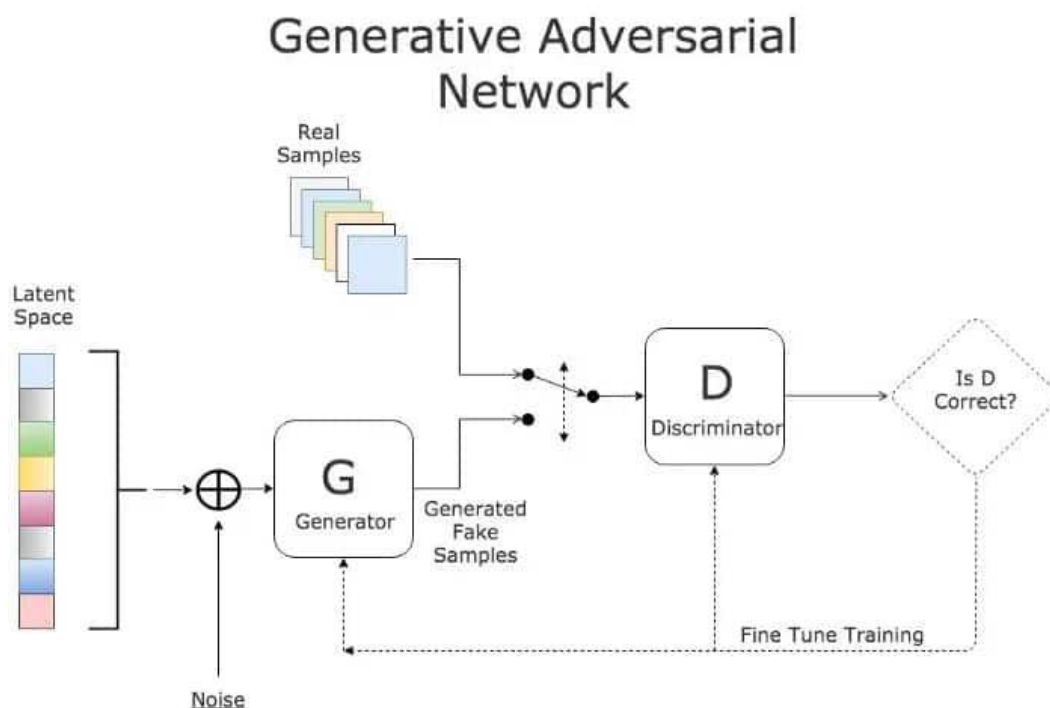
4. Mạng sinh đối nghịch (GAN)

Generative Adversarial Networks (GAN) là một lớp mô hình được thiết kế nhằm tạo ra dữ liệu giả mà có đặc điểm tương tự dữ liệu thực. Tên của mạng này xuất phát từ kiến trúc gồm hai mạng có mục tiêu đối nghịch nhau: Generator và Discriminator. Trong đó, Generator học cách tạo ra dữ liệu giả để đánh lừa mô hình Discriminator, trong khi Discriminator học cách phân biệt giữa dữ liệu giả và dữ liệu thật. Qua quá trình huấn luyện, cả hai mô hình này cùng cải thiện khả năng của mình.

GAN có nhiều ứng dụng phổ biến, bao gồm:

- Tạo khuôn mặt người: GAN có thể được sử dụng để tạo ra hình ảnh khuôn mặt người mới, có thể tổng hợp các đặc điểm từ nhiều hình ảnh khác nhau.

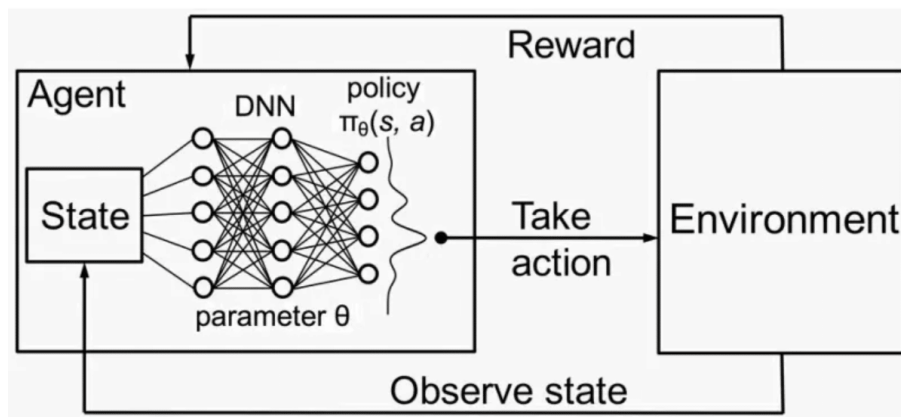
- Thay đổi độ tuổi khuôn mặt: GAN có thể được sử dụng để biến đổi độ tuổi của một khuôn mặt, từ việc tạo ra hình ảnh trẻ hơn đến hình ảnh già hơn.
- Sinh ảnh vật thể: GAN có thể được sử dụng để tạo ra ảnh vật thể, như tạo ra hình ảnh của các loài động vật chưa từng tồn tại.
- Tạo nhân vật hoạt hình: GAN có thể được sử dụng để tạo ra nhân vật hoạt hình mới, với các đặc điểm và nét vẽ độc đáo.



Hình 1. 11: Mô hình hoạt động của mạng GAN.

5. Boltzmann machine

Mạng Boltzmann (Boltzmann machine) là một mô hình mạng không có hướng xác định, trong đó các node của mạng được kết nối với nhau thành một vòng tròn. Mô hình này thường được sử dụng để tạo ra các tham số cho mạng. Các ứng dụng phổ biến của mạng này bao gồm giám sát hệ thống và xây dựng hệ thống khuyến nghị nhị phân.



Hình 1. 12: Mô hình học tăng cường sâu.

6. Học tăng cường sâu

Deep Reinforcement Learning là quá trình mà các tác tử (agent) tương tác với môi trường để thay đổi trạng thái của chính nó và đạt được mục tiêu. Trong quá trình này, các tác tử có khả năng quan sát môi trường và thực hiện hành động phù hợp để tối đa hóa phần thưởng hoặc điểm số.

Mô hình Deep Reinforcement Learning bao gồm một input layer, output layer và nhiều hidden layer khác. Trạng thái của môi trường được đưa vào input layer. Qua quá trình huấn luyện liên tục, mô hình học cách dự đoán điểm số hoặc phần thưởng sau mỗi hành động được thực hiện trong từng trạng thái cụ thể.

Deep Reinforcement Learning được ứng dụng chủ yếu trong nhiều lĩnh vực như game cờ vua, poker, xe tự lái, robot và nhiều ứng dụng khác.

7. Autoencoder

Autoencoder là một trong những kỹ thuật Deep Learning phổ biến nhất hiện nay, có khả năng học các biểu diễn của dữ liệu đầu vào mà không cần nhãn, hay nói cách khác thì mạng này có khả năng học không giám sát (unsupervised learning).

Một số loại autoencoder chính bao gồm:

- Sparse (thưa): Số lượng hidden layer lớn hơn số lượng input layer nhằm hạn chế hiện tượng quá khớp (overfitting). Phương pháp này giới hạn hàm mất mát và ngăn không cho autoencoder lạm dụng tất cả các node có trong mạng.
- Denoising (lọc nhiễu): Một phiên bản input được chuyển thành 0 ngẫu nhiên.
- Contractive: Bổ sung hệ số phạt vào hàm mất mát để hạn chế overfitting trong trường hợp số lượng hidden layer lớn hơn input layer.

-
- Stacked: Xếp chồng nhiều hidden layer lên nhau để tạo thành một mạng autoencoder.

Ứng dụng phổ biến của nó bao gồm phát hiện đặc trưng, xây dựng hệ thống khuyến nghị, bổ sung đặc trưng cho tập dữ liệu,...

8. Backpropagation

Backpropagation (lan truyền ngược) là một kỹ thuật quan trọng trong mạng nơ-ron. Nó cho phép tính toán gradient ngược từ layer cuối cùng đến layer đầu tiên của mạng. Quá trình này bắt đầu bằng việc mạng phân tích các tham số và điều chỉnh chúng thông qua hàm mất mát. Sau đó, giá trị lỗi được tính toán sẽ được lan truyền ngược lại để điều chỉnh các tham số một cách phù hợp.

9. Gradient Descent

Trong Deep Learning và tối ưu hoá, việc tìm giá trị nhỏ nhất hoặc lớn nhất của một hàm số là một thách thức phức tạp, và thường khó có thể đạt được nghiệm tối ưu toàn cục. Thay vào đó, chúng ta thường tìm những điểm cực tiểu địa phương, có thể coi là các nghiệm gần đúng cho bài toán.

Các điểm cực tiểu địa phương được xác định toán học bằng việc giải phương trình đạo hàm bằng 0. Tuy nhiên, trong Machine Learning và Deep Learning, việc giải phương trình đạo hàm bằng 0 là một nhiệm vụ khó khăn. Một phương pháp phổ biến để tiếp cận vấn đề là bắt đầu từ một điểm khởi đầu và sử dụng phép lặp để di chuyển dần tới điểm cần tìm. Phương pháp này được gọi là gradient descent và được áp dụng rộng rãi trong quá trình tối ưu.

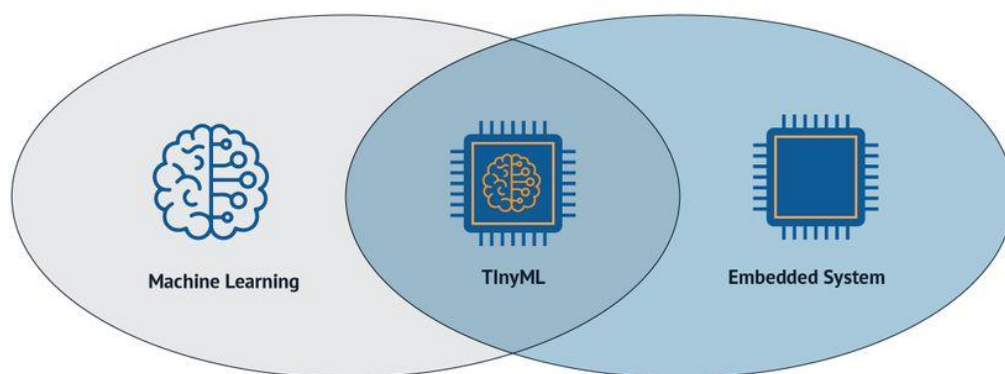
Với sự phát triển của mạng nơ-ron hiện đại, việc sử dụng thuật toán lan truyền ngược (backpropagation) trong gradient descent đã giúp tăng tốc quá trình tối ưu hàng triệu lần so với các phương pháp truyền thống.

1.2.4. Nghiên cứu quy trình, phương pháp triển khai tinyML trên các thiết bị nhúng

1.2.4.1. Khái niệm tinyML

Tiny Machine Learning mang lại khả năng nhận thức cho các thiết bị IoT cực kỳ thiếu tài nguyên như dòng vi điều khiển (Microcontroller units). Thuật ngữ "TinyML" được đặt ra bởi Warden và đồng nghiệp để mô tả các ứng dụng ML sử dụng các mô hình thu nhỏ để thực hiện suy luận vô cùng hiệu quả về năng lượng. Hơn nữa, do thiếu không có Hệ điều hành có tài nguyên phong phú (OS) (ví dụ: Linux, Windows và macOS) trên các MCU, TinyML là một sự kết hợp của ML và các thiết bị IoT hạn chế bộ nhớ và không có Hệ điều hành. Cụ thể, nguồn gốc của TinyML xuất phát từ

sự xuất hiện của EdgeML, giảm nhẹ nhu cầu cơ sở hạ tầng để liên tục truyền/nhận dữ liệu của IoT truyền thống. Tương tự, TinyML không được xem xét là một thay thế cho các mô hình Fog hoặc Cloud; thay vào đó, nó là một thành phần bổ sung hoạt động song song với các mô hình nêu trên.



Hình 1. 13: Sự kết hợp của Machine Learning và Embedded System.

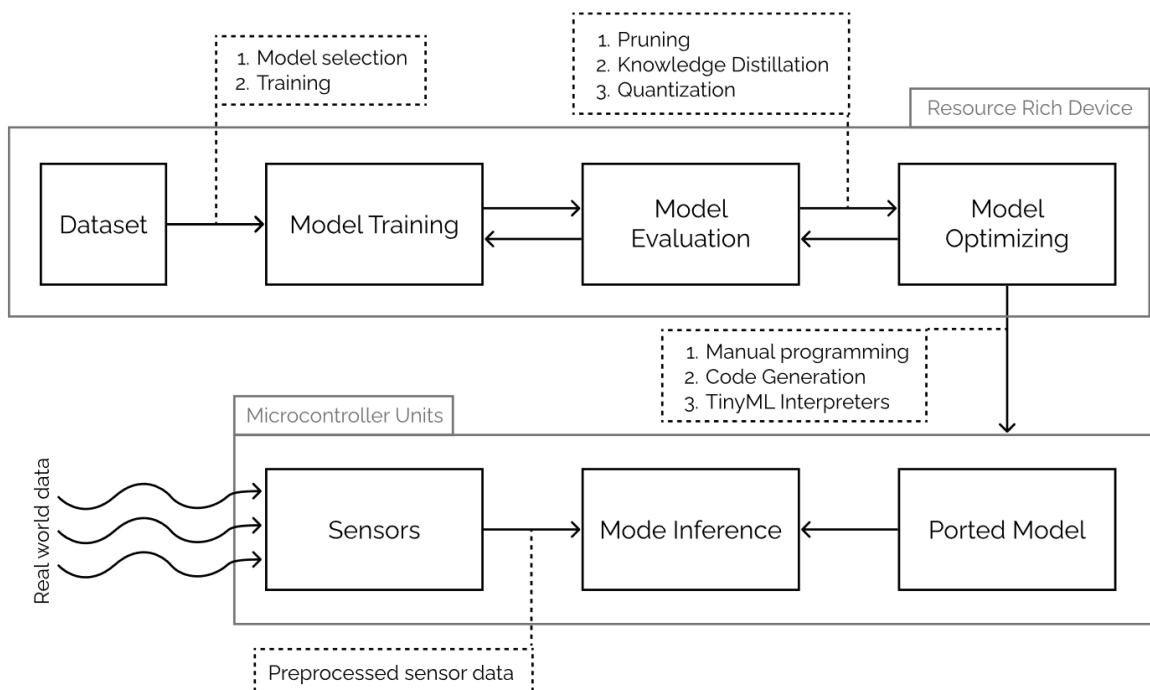
Mức tiêu thụ điện năng của việc triển khai TinyML theo thứ tự milli-Watts (mW). Đây thường là yếu tố phân biệt chúng với các trường hợp học máy nhúng khác. Do đó, việc triển khai TinyML thúc đẩy lợi ích của Edge AI. Cụ thể, chúng có xu hướng vượt trội hơn các triển khai Edge AI khác ở các khía cạnh sau:

- **Độ trễ:** Các hệ thống TinyML không truyền dữ liệu đến bất kỳ máy chủ nào để suy luận, vì các chức năng học máy được thực thi trên chính thiết bị. Do đó, chúng tiết kiệm thời gian truyền dữ liệu, giúp tối ưu hóa độ trễ càng nhiều càng tốt. Ngoài ra, điều này làm cho TinyML rất thích hợp cho một số loại ứng dụng thời gian thực yêu cầu phản hồi ngay lập tức.
- **Tiết kiệm năng lượng:** MCU là thiết bị công suất cực thấp, có thể hoạt động trong thời gian dài mà không cần nguồn năng lượng. Điều này làm giảm năng lượng cần thiết để thực hiện các hoạt động TinyML. Hơn nữa, các hoạt động này tiết kiệm năng lượng đáng kể, do thiếu truyền dữ liệu và hoạt động I / O. Những yếu tố này làm cho việc triển khai TinyML tiết kiệm năng lượng hơn nhiều so với các dạng AI biên khác.

-
- **Tiết kiệm băng thông:** Không giống như các cấu hình Edge AI khác, việc triển khai TinyML không dựa vào kết nối internet để suy luận. Dữ liệu được thu thập và xử lý trên thiết bị thay vì chuyển đến một số máy chủ. Do đó, các hệ thống TinyML cuối cùng cũng cực kỳ hiệu quả về băng thông.
 - **Bảo vệ dữ liệu và quyền riêng tư mạnh mẽ hơn:** Trong phạm vi triển khai TinyML, không có dữ liệu về các nút sưng mù và các máy chủ khác. Điều này khiến các tác nhân độc hại không thể truy cập thông tin nhạy cảm bằng cách hack các máy chủ biên hoặc thiết bị mạng. Nhìn chung, việc thực hiện các mô hình học máy trên một thiết bị nhúng dẫn đến đảm bảo quyền riêng tư dữ liệu mạnh mẽ hơn.
 - **Tăng hiệu quả và tính linh hoạt:** MCU và các thiết bị nhúng khác có kích thước nhỏ hơn máy chủ và máy tính cá nhân. Điều này có nghĩa là chúng đòi hỏi ít không gian và năng lượng hơn nhiều trong khi cung cấp sự linh hoạt triển khai tăng lên. Ví dụ, các hệ thống TinyML cung cấp một trong những cách tốt nhất để triển khai các mô hình học máy ở những nơi không có đủ không gian cho thiết bị cồng kềnh.

1.2.4.2. Quy trình triển khai tinyML trên các thiết bị nhúng

Hệ thống nhúng là các thiết bị đa dạng, từ vài Kilobytes đến vài trăm Kilobytes bộ nhớ, dưới 2 Megabytes bộ nhớ flash và sử dụng một vài miliwatt (<3mW) năng lượng. Các thiết bị di động truyền thống, mặc dù ngày càng có khả năng tính toán và hiệu quả năng lượng, vẫn tiêu thụ từ hàng trăm miliwatt đến vài watt để hoạt động. Các mức sử dụng năng lượng này cao đáng kể so với những gì MCUs tiêu thụ. Quy trình làm việc truyền thống để triển khai TinyML có thể được quan sát trong Hình 1.15.



Hình 1. 14: Quy trình triển khai TinyML trên các thiết bị nhúng.

Quá trình được chia thành ba giai đoạn riêng biệt: Đào tạo, tối ưu hóa và triển khai như sau:

- Bước ban đầu trong việc phát triển một mô hình TinyML là lựa chọn một thuật toán phù hợp và thu thập dữ liệu cho nó. Cơ sở dữ liệu đã được biên soạn trước hoặc dữ liệu cảm biến được thu thập trong quá trình thực hiện có thể được tận dụng để đáp ứng mục đích này. Khi đã có đủ thông tin, quá trình đào tạo mô hình có thể được thực hiện trên một thiết bị có tài nguyên phong phú (ví dụ: máy chủ hoặc máy tính cá nhân). Các công cụ Machine Learning TensorFlow, Pytorch, Scikit-Learn có thể được sử dụng để đào tạo.
- Sau khi quá trình đào tạo mô hình hoàn tất, mô hình kết quả cần được tối ưu hóa. Quá trình tối ưu hóa mô hình có thể được thực hiện thông qua các kỹ thuật như Pruning, Knowledge Distillation, Quantization và mã hóa để đạt được các yêu cầu mong muốn. Pruning là một quá trình lặp lại, một cách có hệ thống để loại bỏ các tham số từ Mạng Nơ-ron (Neural Network - NN). Mục tiêu của Pruning là giảm kích thước của mô hình, loại bỏ các tham số không quan trọng, và làm cho mô hình trở nên hiệu quả hơn trong việc triển khai và suy luận. Knowledge Distillation là quá trình giảng dạy một mô hình nhỏ (student model), được tối ưu hóa, để mô phỏng các kích hoạt của một mô hình đã được huấn luyện trước đó (teacher model). Mục tiêu của KD là chuyển giao kiến thức từ mô hình lớn (teacher) sang mô hình nhỏ (student) để giảm kích thước của mô hình student, nhưng vẫn giữ được hiệu suất tương tự. Quantization là quá trình giảm độ chính

xác số học của mạng nơ-ron, chẳng hạn như biểu diễn trọng số và kích hoạt bằng số nguyên thay vì số chấm động (floating point). Mục tiêu của Quantization là giảm yêu cầu về bộ nhớ và tăng tốc độ suy luận bằng cách giảm độ phức tạp của mô hình.

- Porting và triển khai là các bước cuối cùng trong việc triển khai TinyML. Mô hình học máy cần được đưa về ngôn ngữ MCU có thể hiểu được trước khi triển khai. Thông thường, C/C++ là ngôn ngữ phổ biến cho các ứng dụng như vậy. Có thể sử dụng các trình thông dịch TinyML (ví dụ như Tensorflow Lite Micro) được sử dụng để chuyển đổi một mô hình được xây dựng bằng một ngôn ngữ lập trình có sẵn thành một đồ thị đóng (frozen graph) được biểu diễn lý tưởng dưới dạng một mảng C.

1.2.4.3. Các phương pháp triển khai tinyML trên các thiết bị nhúng.

Sự phổ biến của các thiết bị nhúng giúp thúc đẩy các nghiên cứu về các công nghệ tài nguyên thấp có thể được đưa ứng dụng TinyML. Một thành phần quan trọng của các giải pháp TinyML là cơ chế tự suy luận trên các thiết bị nhúng. Các mô hình được tạo ra bởi các framework ML phổ biến như Tensorflow, Pytorch và SKLearn không phù hợp với TinyML cho các thiết bị nhúng. Do đó, nhiều runtime đã được đề xuất để hỗ trợ ML/DL trên MCUs. Tuy nhiên, chỉ có ba phương pháp khác nhau đã được xác định cho việc triển khai giải pháp TinyML, đó là Lập trình thủ công, Công cụ tạo mã và Trình thông dịch ML như sau:

- Lập trình thủ công (Manual Programming) mang lại kết quả tốt nhất chủ yếu do khả năng tinh chỉnh mọi khía cạnh để tối ưu hóa mục đích chọn lựa. Mặc dù có tiềm năng sản xuất nghiên cứu, việc chia sẻ kiến thức thường bị hạn chế. Điều này là do các tối ưu hóa thực hiện được che giấu và hạn chế cho một bên đã được chọn. Hơn nữa, sự đa dạng của các MCU làm hạn chế khả năng thống nhất của các phương pháp nhúng ML chuyên sâu này. Từ quan điểm chức năng, giải pháp TinyML sẽ có lợi ích về hiệu suất từ phương pháp này nhưng phải đối mặt với chi phí về khả năng tái tạo và thời gian, khiến cho các triển khai như vậy không phải lúc nào cũng là lựa chọn lý tưởng cho những người không chuyên sâu trong lĩnh vực (ví dụ: Người nghiệp dư). Tuy nhiên, từ góc độ có thể tái cấu trúc, sự thiếu hụt các bộ công cụ để cập nhật mô hình ngay lập tức làm cho phương pháp lập trình thủ công là lựa chọn duy nhất.
- Công cụ sinh mã (Code Generation) được ưa chuộng về mặt tiện ích vì một giải pháp tối ưu có thể được đạt được mà không cần lập trình thủ công. Nhiều công cụ tạo mã như EdgeImpulse, Imagimob tận dụng AutoML để cung cấp các công cụ cần thiết dưới dạng Phần mềm như Dịch vụ, trong khi hoạt động như một công cụ

tìm kiếm cho thư viện TinyML của bên thứ ba. Một nhược điểm của các phương pháp tạo mã là thị trường và các nhà cung cấp đã có bộ công cụ và chủ quyền.

Tool	Specified Supported Architectures	Language	Compiler	Runtime	Specialization
EdgeImpulse[40]	Cortex-M7 Cortex-M3 Cortex-M4 Cortex-M4F Cortex-M33 Cortex-M0+	C++	EON	TFLM	(1) Freely available. (2) Prebuilt models cannot be imported but the architectures can. (3) Converts the NN to source code for 20-30% reduction in memory usage & eliminates the need for the TFLM interpreter.
Imagimob[39]	Cortex-M0 Cortex-M3 Cortex-M4	C	N/A	N/A	(1) Subscription based (2) Relies on AutoML based C code generator named Imaginet. (3) Supports post-training quantization for Long-Short Term Memory layers. (4) Eliminates the dependence on run-times as code is statically compiled. (5) Utilize "imunits" for optimizations.
NanoEdge AI[18]	All Cortex-M	C	N/A	N/A	(1) Search engine for static TinyML models (2) Ability to limit memory allocated for the model (3) Not limited to NN implementations

Hình 1. 15: Các công cụ hỗ trợ tạo code tự động.

- Các trình thông dịch(Interpreters) cung cấp tính tiện lợi vì có thể được sử dụng trên mọi môi trường. Các mô hình ML/DL có thể dễ dàng được chuyển đổi vì kiến trúc của mô hình không liên kết với framework. Ngoài ra, các mô hình có thể được tối ưu hóa và điều chỉnh riêng lẻ để phù hợp với nhu cầu và thiết bị. Mặc dù tiện lợi, mô hình này đi kèm với một lượng nhỏ chi phí về hiệu suất và việc sử dụng bộ nhớ. Hơn nữa, sự tách biệt giữa mô hình và quy trình ở cấp hệ thống cho phép có khả năng tổng quát hóa lớn hơn, giúp dễ dàng giới thiệu các tiêu chí đánh giá.

PHẦN 2: THỰC TẬP CHUYÊN SÂU TẠI LAB RẠNG ĐÔNG

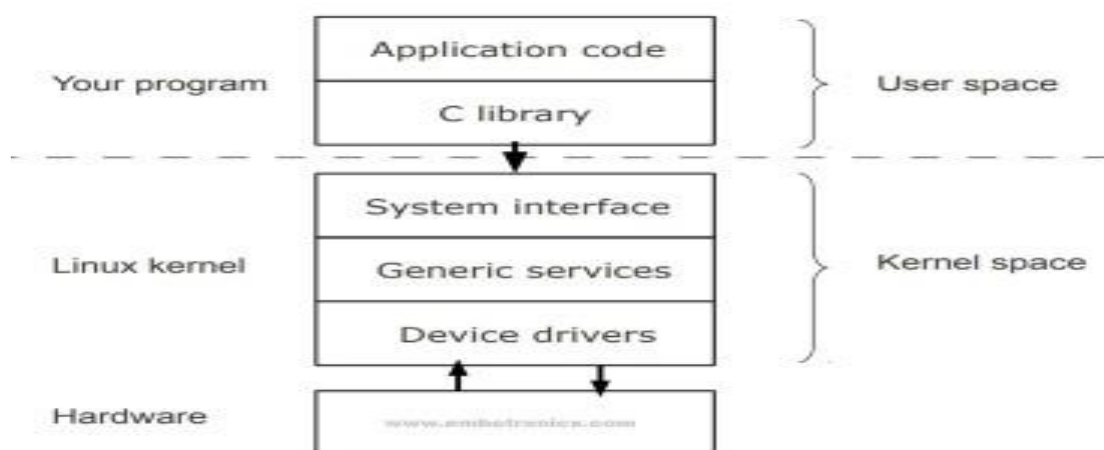
2.1. Tổng quan về hệ điều hành Linux

Hệ điều hành Linux là một hệ điều hành miễn phí và mã nguồn mở, được xây dựng dựa trên kernel Linux. Hệ điều hành này được thiết kế để hoạt động trên một loạt các thiết bị, bao gồm máy tính để bàn, máy tính xách tay, máy chủ, thiết bị nhúng và điện thoại thông minh. Linux được đánh giá cao về tính linh hoạt và khả năng tùy chỉnh, cho phép người dùng điều chỉnh các thành phần để đáp ứng đầy đủ nhu cầu cá nhân. Linux có một loạt các bản phân phối khác nhau, thường được gọi là các phiên bản phân phối (distribution). Các phiên bản phân phối phổ biến bao gồm Ubuntu, Debian, Fedora, CentOS và Red Hat Enterprise Linux. Mỗi phiên bản phân phối mang đến các tính năng, giao diện và tiện ích riêng biệt, tuy nhiên, tất cả đều dựa trên kernel Linux và sử dụng các công cụ phát triển mã nguồn mở. Linux dựa trên tinh thần mã nguồn mở, cho phép người dùng truy cập mã nguồn của hệ thống để tùy chỉnh hoặc phát triển theo ý muốn. Điều này thúc đẩy sự phát triển và cải tiến liên tục từ cộng đồng người dùng, đáp ứng mọi nhu cầu đa dạng. Một trong những điểm đặc biệt của Linux là khả năng bảo mật cao, bởi vì hệ thống được xây dựng với quyền người dùng tối thiểu. Điều này giúp giảm thiểu rủi ro từ các cuộc tấn công của hacker và phần mềm độc hại. Ngoài ra, Linux còn tích hợp hệ thống quản lý gói (package management system), giúp người dùng dễ dàng cài đặt và quản lý phần mềm. Hệ thống này cho phép người dùng dễ dàng cài đặt phần mềm mới và cập nhật chúng một cách an toàn và tiện lợi.

2.1.1. Kiến trúc hệ điều hành Linux

Linux chủ yếu được chia thành User Space & Kernel Space. Hai thành phần này tương tác thông qua Giao diện cuộc gọi hệ thống - là giao diện được xác định trước và hoàn thiện cho Nhân Linux cho các ứng dụng Không gian người dùng. Hình ảnh dưới đây sẽ cung cấp cho bạn một sự hiểu biết cơ bản.

Kernel vs user space



Hình 2. 1: Sơ đồ khối của một hệ thống máy tính

- Kernel Space là nơi hạt nhân (tức là lõi của hệ điều hành) thực thi (tức là chạy) và cung cấp các dịch vụ của nó.
- User Space là nơi các ứng dụng người dùng được thực thi.

2.1.2. Linux Kernel Modules

Linux Kernel Modules là các đoạn mã có thể được tải và dỡ vào hạt nhân theo yêu cầu. Chúng mở rộng chức năng của nhân mà không cần khởi động lại hệ thống. Mã tùy chỉnh có thể được thêm vào nhân Linux thông qua hai phương pháp:

- Cách cơ bản là thêm mã vào cây nguồn kernel và biên dịch lại kernel.
- Một cách hiệu quả hơn để thực hiện điều này là thêm mã vào nhân khi nó đang chạy. Quá trình này được gọi là tải mô-đun, trong đó mô-đun đề cập đến mã mà chúng tôi muốn thêm vào nhân.

Vì các mã này trong thời gian chạy và chúng không phải là một phần của nhân Linux chính thức, nên chúng được gọi là loadable kernel module (LKM), khác với “base kernel”. Nhân cơ sở nằm trong thư mục /boot và luôn được tải khi chúng ta khởi động máy trong khi các LKM được tải sau khi nhân cơ sở đã được tải. Tuy nhiên, LKM này là một phần rất quan trọng trong nhân của chúng tôi và chúng giao tiếp với nhân cơ sở để hoàn thành các chức năng của chúng.

LKM có thể thực hiện nhiều nhiệm vụ khác nhau, nhưng về cơ bản, chúng có ba loại chính:

-
- Device driver: trình điều khiển thiết bị được thiết kế cho một phần cứng cụ thể. Nhân sử dụng nó để giao tiếp với phần cứng đó mà không cần biết bất kỳ chi tiết nào về cách thức hoạt động của phần cứng.
 - Filesystem drivers: Trình điều khiển hệ thống tệp diễn giải nội dung của hệ thống tệp (thường là nội dung của ổ đĩa) dưới dạng tệp và thư mục, v.v. Có rất nhiều cách khác nhau để lưu trữ tệp và thư mục, v.v. trên ổ đĩa, trên máy chủ mạng và theo những cách khác. Đối với mỗi cách, bạn cần một trình điều khiển hệ thống tệp tin. Ví dụ: có một trình điều khiển hệ thống tệp cho loại hệ thống tệp ext2 được sử dụng hầu như phổ biến trên các ổ đĩa Linux. Có một cái cho hệ thống tệp MS-DOS và một cái cho NFS.
 - System call: Các chương trình không gian người dùng sử dụng các cuộc gọi hệ thống để nhận các dịch vụ từ kernel. Ví dụ: có các lệnh gọi hệ thống để đọc một tệp, để tạo một quy trình mới và để tắt hệ thống. Hầu hết các cuộc gọi hệ thống là không thể thiếu đối với hệ thống và rất chuẩn, vì vậy luôn được tích hợp vào nhân cơ sở (không có tùy chọn LKM).

Sự khác biệt giữa kernel module và user programs:

- Kernel modules có không gian địa chỉ riêng biệt. Một mô-đun chạy trong không gian hạt nhân. Một ứng dụng chạy trong không gian người dùng. Phần mềm hệ thống được bảo vệ khỏi các chương trình người dùng. Không gian hạt nhân và không gian người dùng có không gian địa chỉ bộ nhớ riêng.
- Kernel modules có đặc quyền thực thi cao hơn. Mã chạy trong không gian kernel có đặc quyền lớn hơn mã chạy trong không gian người dùng.
- Kernel modules không thực thi tuần tự. Một chương trình người dùng thường thực hiện tuần tự và thực hiện một nhiệm vụ duy nhất từ đầu đến cuối. Mô-đun hạt nhân không thực thi tuần tự. Một mô-đun hạt nhân tự đăng ký để phục vụ các yêu cầu trong tương lai.
- Kernel modules sử dụng các tệp tiêu đề khác nhau. Các mô-đun hạt nhân yêu cầu một tập hợp các tệp tiêu đề khác với các chương trình người dùng yêu cầu.

Sự khác biệt giữa kernel module và kernel driver:

- Kernel module là một đoạn mã được biên dịch có thể được chèn vào hạt nhân trong thời gian chạy, chẳng hạn như withinsmod hoặc modprobe.
- Driver là một đoạn mã chạy trong nhân để giao tiếp với một số thiết bị phần cứng. Nó “điều khiển” phần cứng. Hầu như mọi phần cứng trong máy tính của bạn đều có trình điều khiển liên quan.

2.1.3. Linux driver

Driver là một phần mềm, gồm các lệnh, hướng dẫn CPU cách tương tác với thiết bị. Các thiết bị có thể là chuột, bàn phím, ổ cứng, card mạng, loa, màn hình,... Tuy nhiên, các thiết bị này không được nối trực tiếp với CPU, bởi vì:

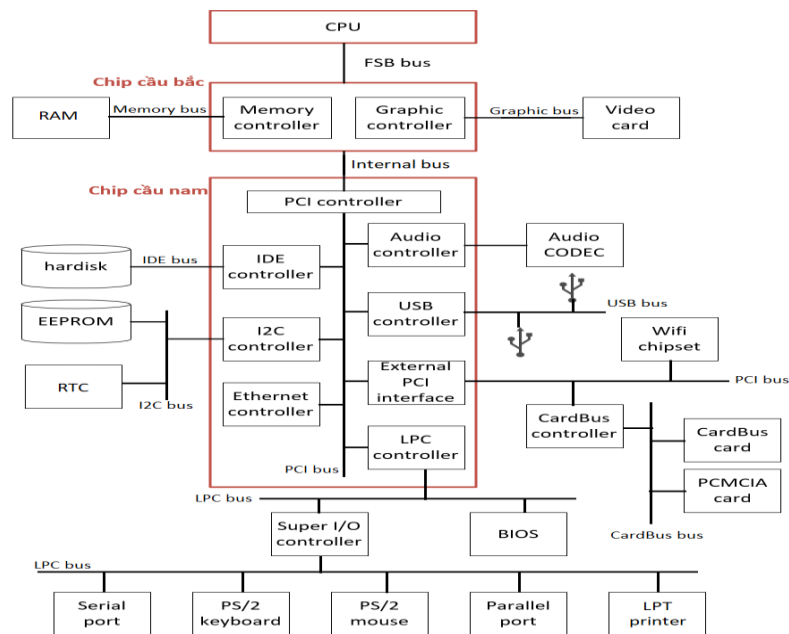
- Hệ thống có nhiều thiết bị, nhưng số lượng chân của CPU hữu hạn.
- Tốc độ làm việc của các thiết bị thấp hơn nhiều so với CPU.

Chính vì vậy, các thiết bị kết nối với CPU thông qua một thiết bị khác, gọi là bộ điều khiển (device controller). Có nhiều cách để phân loại bộ điều khiển:

Phân loại theo chức năng, ví dụ:

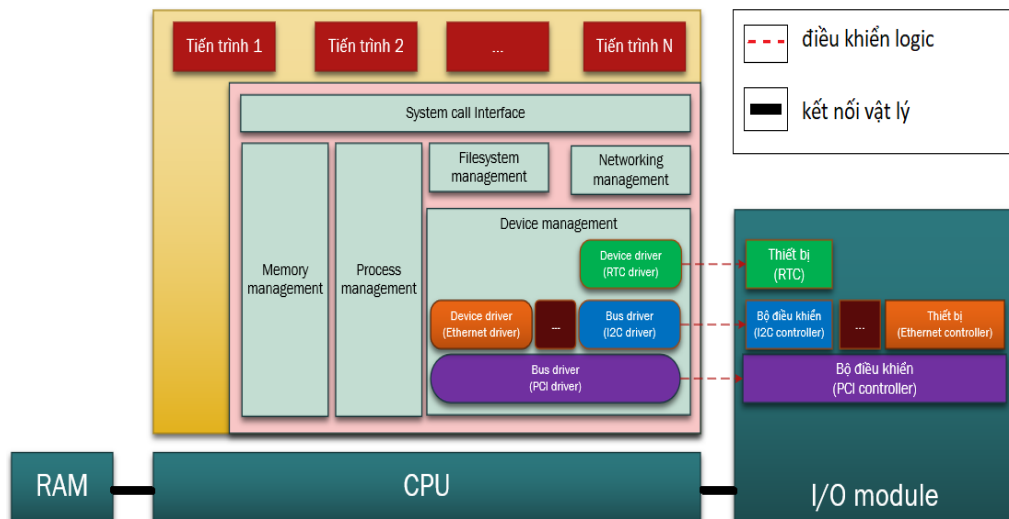
- Hard disk controller: trợ giúp CPU điều khiển ổ cứng.
- Graphic controller: trợ giúp CPU điều khiển các thiết bị hiển thị.
- Keyboard controller: trợ giúp CPU điều khiển bàn phím.
- Phân loại theo kỹ thuật giao tiếp với các thiết bị, ví dụ:
- PCI controller: hỗ trợ CPU giao tiếp với các thiết bị khác, mà các thiết bị này được thiết kế theo chuẩn PCI.
- USB controller: hỗ trợ CPU giao tiếp với các thiết bị khác, mà các thiết bị này được thiết kế theo chuẩn USB.
- I2C controller: hỗ trợ CPU giao tiếp với các thiết bị khác, mà các thiết bị này được thiết kế theo chuẩn I2C.

Hình 2.2 là sơ đồ khối của hệ thống máy tính cá nhân, sử dụng bộ xử lý của Intel. Từ hình này, ta cũng thấy rằng, thiết bị (device) và bộ điều khiển (device controller) chỉ mang tính chất tương đối. Ví dụ, trên PCI bus, PCI controller đóng vai trò là bộ điều khiển, còn I2C controller đóng vai trò là thiết bị. Nhưng trên I2C bus, thì I2C controller lại là bộ điều khiển, còn EEPROM và RTC đóng vai trò là thiết bị.



Hình 2. 2: Sơ đồ khối của một hệ thống máy tính.

Đứng từ góc độ của CPU, bộ điều khiển cũng chỉ là một thiết bị. Do đó, cần có driver hướng dẫn CPU làm việc với bộ điều khiển. Driver này được gọi là bus driver. Còn driver hướng dẫn CPU làm việc với thiết bị thì được gọi là device driver.



Hình 2. 3: Mối liên hệ giữa Driver, bộ điều khiển và thiết bị dưới góc độ thực thi.

Để thuận tiện cho quá trình phát triển, bus driver và device driver được chia làm 2 thành phần (Hình 2.3).

Device driver gồm 2 phần:

-
- Thành phần OS specific. Thành phần này cung cấp cho hệ điều hành các dịch vụ đọc/ghi dữ liệu của thiết bị. Điều này cho phép chúng ta xây dựng hệ điều hành độc lập với cấu trúc của thiết bị.
 - Thành phần device specific. Thành phần này chứa các lệnh hướng dẫn CPU điều khiển thiết bị, giám sát thiết bị, trao đổi dữ liệu với thiết bị. Chúng ta sử dụng datasheet của thiết bị để xây dựng thành phần này. Datasheet là một tài liệu được cung cấp bởi nhà sản xuất thiết bị. Nó mô tả sơ đồ khối chức năng, nguyên lý hoạt động, hiệu suất hoạt động, đặc tính điện của thiết bị và đặc biệt là bản đồ thanh ghi (register map).

Bus driver gồm 2 phần:

- Thành phần protocol abstraction. Thành phần này che giấu đi sự phức tạp của các giao thức trên bus, cung cấp các dịch vụ cho device driver sử dụng. Ví dụ như đọc/ghi một thanh ghi nào đó của thiết bị.
- Thành phần protocol specific. Thành phần này chứa các lệnh hướng dẫn CPU làm việc với bộ điều khiển, giúp đọc/ghi dữ liệu trên bus. Nó cũng được xây dựng dựa trên datasheet của bộ điều khiển.

Linux device driver:

Linux device driver là một phần mềm cho phép hệ thống Linux tương tác với các thiết bị phần cứng. Nó cung cấp giao tiếp giữa kernel và các thiết bị phần cứng, cho phép các ứng dụng sử dụng các tính năng của thiết bị.

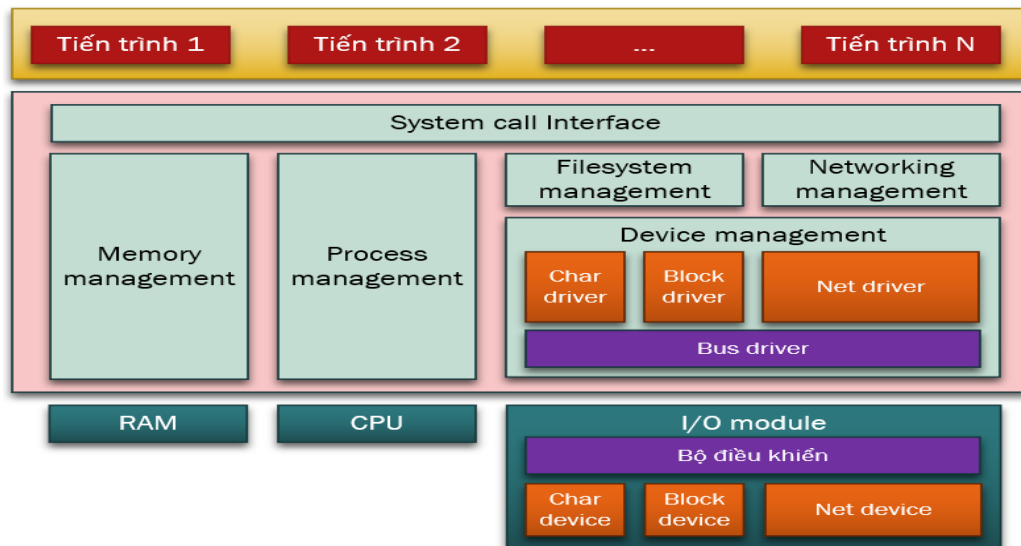
Linux device driver được sử dụng rộng rãi trong các ứng dụng nhúng và thiết bị phần cứng. Nó cung cấp cho các nhà sản xuất thiết bị một cách để tối ưu hóa hiệu suất của thiết bị của họ, tăng khả năng tương thích và cải thiện tính ổn định của thiết bị.

Phân loại:

- Character device driver là một loại driver trong hệ điều hành, cho phép các chương trình ứng dụng truy cập vào các thiết bị đầu cuối như bàn phím, chuột, màn hình hiển thị, hoặc các thiết bị ngoại vi khác. Nó giúp cho các chương trình ứng dụng có thể tương tác với thiết bị phần cứng và truyền dữ liệu giữa chúng.
- Block device driver là một loại driver cho phép quản lý các thiết bị lưu trữ dưới dạng các khối (block) như ổ đĩa cứng, USB, ổ đĩa CD-ROM và DVD-ROM, các thiết bị đọc thẻ nhớ và các thiết bị khác. Block device driver cho phép truy xuất và quản lý các phân vùng đĩa cứng, tạo ra các hệ thống tập tin,

tạo ra các hệ thống RAID, cung cấp cơ chế lưu trữ đệm và thống kê hiệu suất của các thiết bị lưu trữ.

- Network device driver là một loại driver cho phép quản lý các thiết bị mạng như card mạng Ethernet, card mạng không dây, card mạng Bluetooth và các thiết bị mạng khác. Network device driver cho phép truyền và nhận dữ liệu trên các kênh mạng, kiểm soát gói tin, xử lý lỗi, phân tích hiệu suất và thực hiện các thao tác mạng khác.



Hình 2. 4: Các thành phần của Bus Driver và Device Driver.

2.2. Tạo hệ điều hành linux sử dụng nền tảng mã nguồn mở Openwrt

2.2.1. Tổng quan về OpenWRT

OpenWrt là một dự án mã nguồn mở cho các hệ điều hành nhúng dựa trên Linux, chủ yếu được sử dụng trên các thiết bị nhúng để định tuyến lưu lượng mạng. Các thành phần chính là Linux, use -linux, musl, và BusyBox. Tất cả các thành phần đã được tối ưu hóa để đủ nhỏ để phù hợp với bộ nhớ và lưu trữ hạn chế có sẵn trong các bộ định tuyến (thường chỉ giao động từ 4-16MB) và hoạt động trong môi trường cấu hình thấp (với CPU chỉ vài trăm Mhz và RAM chỉ từ 32MB). OpenWrt được cấu hình bằng giao diện dòng lệnh (ash shell) hoặc giao diện web (LuCI). Có khoảng 3500 gói phần mềm tùy chọn có sẵn để cài đặt thông qua hệ thống quản lý gói opkg.

OpenWrt được thiết kế bởi các chuyên gia mạng và kết hợp nhiều thuật toán điều khiển bộ đệm gần đây để thực hiện định tuyến. Điều này có nghĩa là ngoài tính ổn định, OpenWrt còn giảm độ trễ và tăng thông lượng mạng. OpenWrt cũng mang đến những cải tiến cho bảo mật mạng, với các bản cập nhật thường xuyên để đóng các lỗ

hồng bảo mật. Tổng cộng, OpenWrt cung cấp hơn 3000 gói sẵn sàng được cài đặt, dễ dàng thêm các chức năng vào bộ định tuyến, bao gồm:

- Chặn quảng cáo trực tiếp trên bộ định tuyến.
- Mã hóa kết nối internet của bạn để bảo mật hơn.
- Đặt hạn ngạch về khối lượng tải xuống hoặc băng thông.
- Tạo mạng khách để cho phép truy cập internet nhưng không cho phép các thiết bị cục bộ.
- Thiết lập bộ định tuyến làm trung tâm cho việc tự động hóa gia đình.
- Giám sát mạng thời gian thực.
- Tạo DNS động.
- Thiết lập máy khách hoặc máy chủ VPN.
- Chạy ứng dụng khách BitTorrent từ bộ định tuyến.

Giống như các hệ điều hành Linux khác, OpenWrt có khả năng cấu hình cực cao, vì vậy bạn chỉ có thể đưa những gì bạn muốn cho bộ định tuyến của mình.

2.2.2. Xây dựng hệ điều hành

➤ Cài môi trường:

<code>apt-get update</code>
<code>apt-get upgrade</code>
<code>apt install build-essential ccache ecj fastjar file g++ gawk gettext git java-propose-classpath libelf-dev libncurses5-dev libncursesw5-dev libssl-dev python2.7-dev python3 unzip wget python3-distutils python3-setuptools python3-dev rsync subversion swig time xsltproc zlib1g-dev</code>

➤ Kéo gói trên Openwrt về:

<code>git clone https://git.openwrt.org/openwrt/openwrt.git</code>
<code>cd openwrt</code>

➤ Kiểm tra phiên bản:

<code>git pull</code>

git branch -a
git checkout v21.02.3

```
root@8b78de80a1fb:/home/backup/openwrt# git pull
Already up to date.
root@8b78de80a1fb:/home/backup/openwrt# git branch -a
* master
remotes/origin/HEAD → origin/master
remotes/origin/lede-17.01
remotes/origin/main
remotes/origin/master
remotes/origin/openwrt-18.06
remotes/origin/openwrt-19.07
remotes/origin/openwrt-21.02
remotes/origin/openwrt-22.03
remotes/origin/openwrt-23.05
root@8b78de80a1fb:/home/backup/openwrt# git checkout v21.02.3
Note: switching to 'v21.02.3'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 42a15ca378 OpenWrt v21.02.3: adjust config defaults
root@8b78de80a1fb:/home/backup/openwrt#
```

Hình 2. 5: Kéo gói Openwrt về

➤ Cập nhật lại các gói:

./scripts/feeds update -a
./scripts/feeds install -a

➤ Tích các gói trong Menuconfig:

make menuconfig

➤ Chọn gói cấu hình giao diện mạng:

```
1. Collections
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters
are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?>
for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable

[*] luci..... LuCI interface with Uhttpd as Webserver (default)
[*] luci-light..... LuCI interface with Uhttpd as Webserver (light)
< > luci-nginx..... LuCI interface with Nginx as Webserver (NEW)
< > luci-ssl..... LuCI with HTTPS support (mbedtls as SSL backend)
< > luci-ssl-nginx. LuCI with HTTPS support on Nginx (OpenSSL as SSL backend) (NEW)
< > luci-ssl-openssl..... LuCI with HTTPS support (OpenSSL as SSL backend) (NEW)

<Select> < Exit > < Help > < Save > < Load >
```

Hình 2. 6: Chọn gói hỗ trợ giao diện web Luci

- Chọn gói hỗ trợ cho giao thức MQTT:

```
Network
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters
are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?>
for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable

^(-)
< > mini_snmpd..... A tiny SNMP server for embedded systems (NEW)
< > minimalist-pcproxy..... Lightweight PCP proxy (NEW)
< > miredo..... Teredo IPv6 tunneling utility (NEW)
< > modemmanager..... Control utility for any kind of mobile broadband modem (NEW)
< > mosquitto-client-nossl.... mosquitto - client tools (Without SSL support) (NEW)
[*] mosquitto-client-ssl..... mosquitto - client tools (With SSL support)
< > mosquitto-nossl. mosquitto - an MQTT message broker (without SSL support) (NEW)
< > mosquitto-ssl..... mosquitto - an MQTT message broker (with SSL support)
[*] libwebsockets support (NEW)
[*] Include mosquitto_ctrl utility (NEW)
[*] Install mosquitto_dynamic_security.so (NEW)
[*] Include mosquitto_passwd utility (NEW)
< > mstpd..... Multiple Spanning Tree Protocol daemon (NEW)
< > mtr-json..... Full screen ncurses traceroute tool With JSON (NEW)
< > mtr-nojson..... Full screen ncurses traceroute tool Without JSON (NEW)
< > natmap..... TCP/UDP port mapping tool for full cone NAT (NEW)
v(+)

<Select> < Exit > < Help > < Save > < Load >
```

Hình 2. 7: Chọn gói hỗ trợ giao thức MQTT

- Chọn gói Zerotier để SSH từ xa:

```

      VPN
      Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
      are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?>
      for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

      ~(-)
      <> softethervpn5-server..... softethervpn5 v5.02.5180 server (NEW)
      <> stp-client..... SSTP-Client is a SSTP client for Linux. (NEW)
      <> strongswan..... StrongSwan (NEW) ----
      <> tailscale..... Zero config VPN (utility) (NEW)
      <> tailscaled..... Zero config VPN (daemon) (NEW)
      <> tinc..... VPN tunneling daemon (NEW)
      <> manytun..... micro anycast tunneling daemon (gcrpt) (NEW)
      <> manytun-nettle..... micro anycast tunneling daemon (nettle) (NEW)
      <> manytun-nocrypt..... micro anycast tunneling daemon (no crypt) (NEW)
      <> manytun-sslcrypt..... micro anycast tunneling daemon (openssl) (NEW)
      <> vpnc..... VPN client for Cisco EasyVPN (NEW)
      <> vpnc-scripts..... VPN configuration script for vpnc and OpenConnect (NEW)
      <> wireguard-tools..... WireGuard userspace control program (wg) (NEW)
      <> xl2tpd..... An L2TP (Layer 2 Tunneling Protocol) daemon (NEW)
      [*] zerotier... Create flat virtual Ethernet networks of almost unlimited size
      configuration --->

      <Select> < Exit > < Help > < Save > < Load >
  
```

Hình 2. 8: Chọn gói zerotier hỗ trợ ssh từ xa

- Chọn gói hỗ trợ cho C++:

```

      Base system
      Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
      are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?>
      for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

      ~(-)
      <> ead..... Emergency Access Daemon (NEW)
      <> firewall..... OpenWrt C Firewall (NEW)
      <> firewall4..... OpenWrt 4th gen firewall (NEW)
      *- fstools..... OpenWrt filesystem tools --->
      *- fwtool
      *- getrandom..... OpenWrt getrandom system helper
      *- jsonfilter..... OpenWrt JSON filter utility
      *- libatomic..... Atomic support library
      *- libc..... C library
      *- libgcc..... GCC support library
      <> libgomp..... OpenMP support library (NEW)
      *- libpthread..... POSIX thread library (NEW)
      *- librt..... POSIX.1b RealTime extension library
      [*] libstdc++..... GNU Standard C++ Library V3
      <> logd..... OpenWrt system log implementation (NEW)
      <> mtd..... Update utility for trx firmware images (NEW)
      v(+)

      <Select> < Exit > < Help > < Save > < Load >
  
```

Hình 2. 9: Chọn thư viện hỗ trợ C++

- Chọn gói hỗ trợ cho định dạng Json:

```

      Libraries
      Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
      are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?>
      for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

      ~(-)
      <> icu-data-tools..... ICU Data manipulation tools (NEW)
      <> icu-full-data..... Full ICU Data (NEW)
      *- jansson..... Jansson library
      <> json-glib..... JSON GLib Library
      [*] jsoncpp..... jsoncpp
      <> knot-libs..... Knot DNS common DNS and DNSSEC libraries (NEW)
      <> knot-libzscanner..... Knot DNS zone parser library (NEW)
      <> libaio..... Linux kernel AIO interface access library (NEW)
      <> libantlr3c..... libantlr3c (NEW)
      <> libao..... A cross platform audio library (NEW)
      <> libapparmor..... AppArmor library (NEW)
      <> libapr..... Apache Portable Runtime Library (NEW)
      <> libaprutil..... Apache Portable Runtime Utility Library (NEW)
      <> libarchive..... Multi-format archive and compression library (NEW)
      <> libarchive-noopenssl (NEW)
      <> libasm..... ELF manipulation libraries (libasm) (NEW)
      v(+)

      <Select> < Exit > < Help > < Save > < Load >
  
```

Hình 2. 10: Chọn gói jsoncpp hỗ trợ chuỗi json

- Lưu cấu hình và chạy lệnh để build:

```
export FORCE_UNSAFE_CONFIGURE=1
```

```
make -j24 V=99
```

➤ Quá trình build hoàn tất:

```
et-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.bin" -s "/home/b
ackup/openwrt/key-build" -x "/home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt
76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.bin.sig" ; ucrt -A -c "/home/backup/openwrt/build_dir/target-mipsel_24kc_mus
l/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.bin.ucrt" -x "/home/backup/openwr
t/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.
bin.sig" ; fwtool -S "/home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-me
diatek_linkit-smart-7688-ext4-sysupgrade.bin.ucrt" "/home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76
x8/tmp/openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.bin" ; }
WARNING: Image file /home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-medi
atek_linkit-smart-7688-ext4-sysupgrade.bin is too big: 10487774322 > 33226752
gzip -c -9n /home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-mediatek_lin
kit-smart-7688-ext4-sysupgrade.bin > /home/backup/openwrt/bin/targets/ramips/mt76x8/openwrt-ramips-mt76x8-mediatek_linkit-sma
rt-7688-ext4-sysupgrade.bin.gz
gzip: /home/backup/openwrt/build_dir/target-mipsel_24kc_musl/linux-ramips_mt76x8/tmp/openwrt-ramips-mt76x8-mediatek_linkit-sm
art-7688-ext4-sysupgrade.bin: No such file or directory
make[5]: *** [Makefile:206: /home/backup/openwrt/bin/targets/ramips/mt76x8/openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-e
xt4-sysupgrade.bin.gz] Error 1
make[5]: Leaving directory '/home/backup/openwrt/target/linux/ramips/image'
make[4]: *** [Makefile:23: install] Error 2
make[4]: Leaving directory '/home/backup/openwrt/target/linux/ramips'
make[3]: *** [Makefile:11: install] Error 2
make[3]: Leaving directory '/home/backup/openwrt/target/linux'
time: target/linux/install#711.91#111.68#302.11
ERROR: target/linux failed to build.
make[2]: *** [target/Makefile:25: target/linux/install] Error 1
make[2]: Leaving directory '/home/backup/openwrt'
make[1]: *** [target/Makefile:19: /home/backup/openwrt/staging_dir/target-mipsel_24kc_musl/stamp/.target_install] Error 2
make[1]: Leaving directory '/home/backup/openwrt'
make: *** [/home/backup/openwrt/include/toplevel.mk:230: world] Error 2
root@f9d79fd30362:/home/backup/openwrt#
```

Hình 2. 11: Quá trình build hoàn tất

➤ Sau khi build xong thì chúng ta tải file image về “openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-squashfs-sysupgrade.bin”:

```
root@f3a9a0d5d08e:/home/backup/openwrt/bin/targets/ramips/mt76x8# ls
config.buildinfo
feeds.buildinfo
openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-ext4-sysupgrade.bin.gz
openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-initramfs-kernel.bin
openwrt-ramips-mt76x8-mediatek_linkit-smart-7688-squashfs-sysupgrade.bin
openwrt-toolchain-ramips-mt76x8_gcc-8.4.0_musl.Linux-x86_64.tar.bz2
packages
version.buildinfo
root@f3a9a0d5d08e:/home/backup/openwrt/bin/targets/ramips/mt76x8#
```

Hình 2. 12: Chọn file image

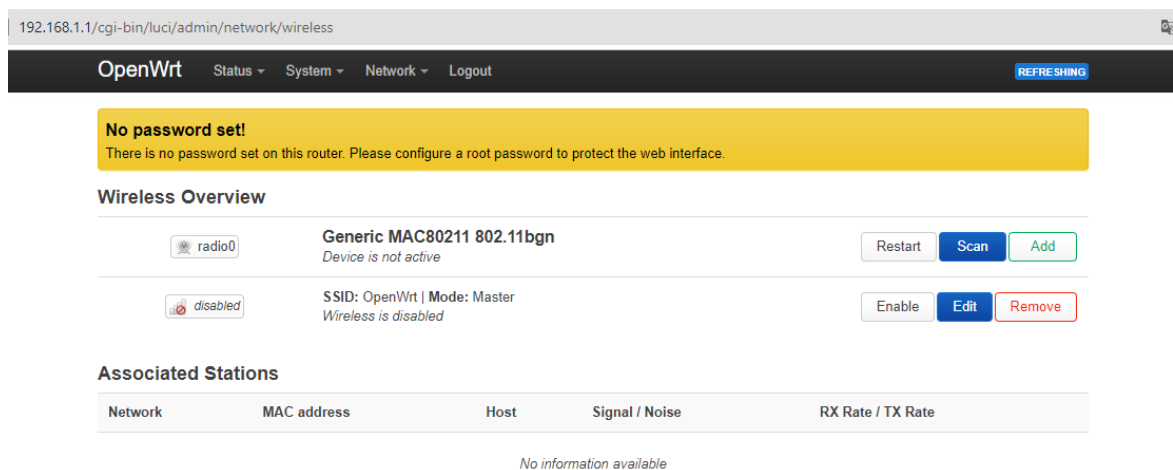
➤ Các bước nạp image cho AI7688H:

```
mount -t vfat /dev/sda1 /root
```

```
cd /root
```

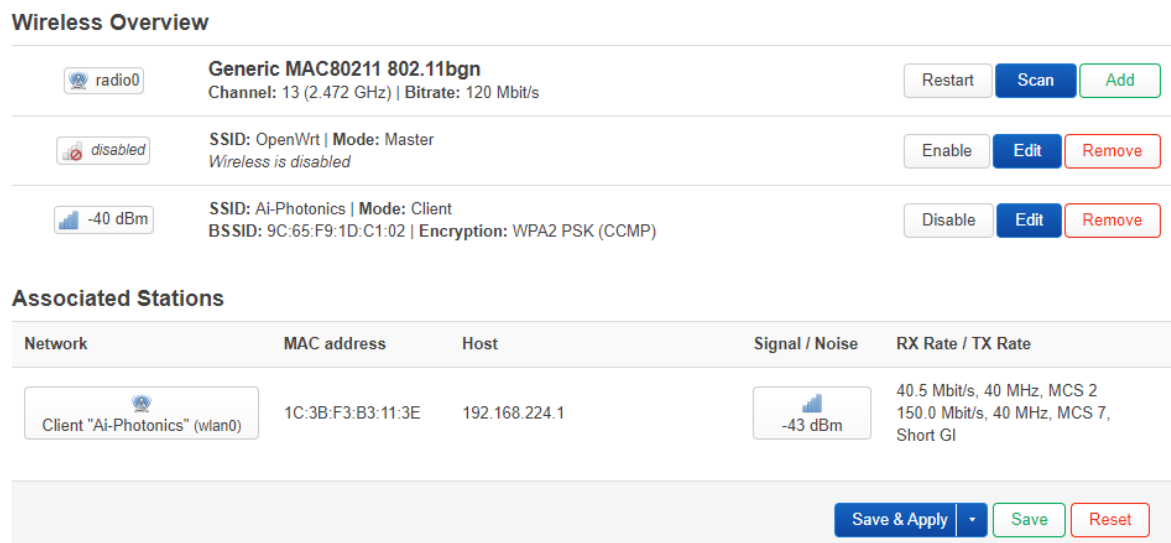
```
sysupgrade -n lks7688.img
```

➤ Quá trình nạp image được hoàn tất:



Hình 2. 15: Kết nối wifi cho board Ai7688.

- Tại Network chọn Wireless và Save , sau đó Save & Apply:



Hình 2. 16: Lưu lại truy cập wifi.

- Tại System chọn Administration:

No password set!

There is no password set on this router. Please configure a root password to protect the web interface.

Router Password

SSH Access

SSH-Keys

SSH Access

Dropbear offers [SSH](#) network shell access and an integrated [SCP](#) server

Dropbear Instance

Delete

Interface

☒ Listen only on the given interface or, if unspecified, on all

Port

Password authentication ☒

☒ Allow [SSH](#) password authentication

Allow root logins with password ☒

☒ Allow the *root* user to login with password

Gateway Ports ☒

☒ Allow remote hosts to connect to local SSH forwarded ports

Add instance

Save & Apply

Save

Reset

Hình 2. 17: Chọn Gateway ports

➤ Tại Network chọn Firewall:

Firewall - Traffic Rules - Unnamed rule

[General Settings](#)[Advanced Settings](#)[Time Restrictions](#)

Name

Protocol

Source zone

Source address

Source port

Destination zone

Destination address

Destination port

Action

Dismiss

Save

Hình 2. 18: Thêm cấu hình ssh

➤ Check lại IP để SSH:

```

UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:20311 errors:0 dropped:0 overruns:0 frame:0
TX packets:28553 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:4386458 (4.1 MiB)  TX bytes:27944285 (26.6 MiB)

eth0      Link encap:Ethernet  HWaddr 9C:65:F9:1D:C0:13
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:20954 errors:0 dropped:0 overruns:0 frame:0
          TX packets:28852 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4705534 (4.4 MiB)  TX bytes:27962601 (26.6 MiB)
          Interrupt:5

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:1174 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1174 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:95350 (93.1 KiB)  TX bytes:95350 (93.1 KiB)

wlan0     Link encap:Ethernet  HWaddr 9C:65:F9:1D:C1:02
          inet addr:192.168.224.102 Bcast:192.168.224.255  Mask:255.255.255.0
          inet6 addr: fe80::9e65:f9ff:fe1d:c102/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:25045 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12294 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:26417642 (25.1 MiB)  TX bytes:3467627 (3.3 MiB)

root@OpenWrt:/#

```

Hình 2. 19: Check IP để ssh (cùng mạng)

Như vậy là chúng ta đã cấu hình SSH xong.

2.2.4. Cài Zerotier cho AI7688H

➤ Cập nhật lại các gói:

```
opkg update
```

➤ Cấu hình file zerotier:

```

nano /etc/config/zerotier

config zerotier 'sample_config'

option enabled '1'

list join '52b337794f81613b'

```

➤ Cấu hình file firewall:

```

nano /etc/config/firewall

config zone 'vpn_zone'

option name 'zerotier'

option input 'ACCEPT'

option forward 'REJECT'

option output 'ACCEPT'

option device 'zt+'

```

```
option masq '1'
option mtu_fix '1'
config forwarding
option dest 'zerotier'
option src 'lan'
config forwarding
option dest 'lan'
option src 'zerotier'
```

- Chạy 2 lệnh sau:

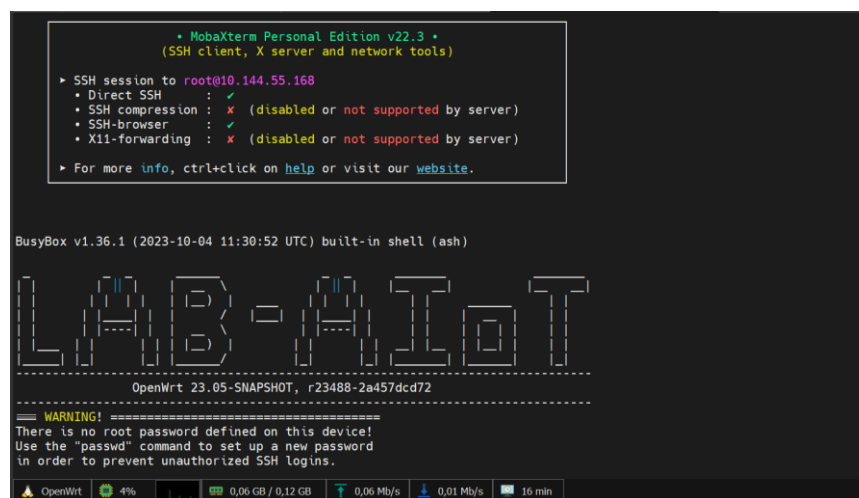
```
/etc/init.d/zerotier restart
```

```
/etc/init.d/firewall restart
```

- Tìm địa chỉ IP để SSH:

```
ifconfig
```

- SSH đến địa chỉ ip 10.144.55.168:



Hình 2. 20: SSH từ xa thông qua zerotier

2.2.5. Cross Compiler

- Tại thư mục backup, tạo thư mục crosscompile:

```
mkdir crosscompile
```

```
root@96b961809d6f:/home/backup# ls
buildroot.sh  crosscompile  openwrt  openwrt.zip  setup.sh
root@96b961809d6f:/home/backup#
```

Hình 2. 21: Tạo thư mục để crosscompile

- Copy thư mục staging_dir vào thư mục crosscompile:

```
cp -r /home/backup/openwrt/staging_dir/ /home/backup/crosscompile/
```

```
root@96b961809d6f:/home/backup/crosscompile# ls
led_gpio11.c  mqtt_ai7688  mqtt_ai7688_json  mt76x8_gpio.c  staging_dir
root@96b961809d6f:/home/backup/crosscompile#
```

Hình 2. 22: Thư mục chứa staging_dir

- Chạy thêm các lệnh sau:

```
cd /staging_dir/toolchain-mipsel_24kc_gcc-8.4.0_musl
```

```
cp -r ../target-mipsel_24kc_musl/usr/* .
```

```
export STAGING_DIR=/home/backup/crosscompile/staging_dir
```

```
export TOOLCHAIN_DIR=$STAGING_DIR/toolchain-mipsel_24kc_gcc-8.4.0_musl
```

```
export LDCFLAGS=$TOOLCHAIN_DIR/lib
```

```
export PATH=$TOOLCHAIN_DIR/bin:$PATH
```

```
export PATH=$PATH: /home/backup/openwrt/staging_dir/toolchain-mipsel_24kc_gcc-8.4.0_musl/bin
```

- Kiểm tra toolchain:

```
root@f9d79fd30362:/home/backup/crosscompile# ls
applications led_gpio11.c mqtt_ai7688 mqtt_ai7688_json staging_dir
root@f9d79fd30362:/home/backup/crosscompile# mipsel-openwrt-linux-
mipsel-openwrt-linux-addr2line mipsel-openwrt-linux-ld.bfd mipsel-openwrt-linux-musl-ld
mipsel-openwrt-linux-ar mipsel-openwrt-linux-musl-addr2line mipsel-openwrt-linux-musl-ld.bfd
mipsel-openwrt-linux-as mipsel-openwrt-linux-musl-ar mipsel-openwrt-linux-musl-nm
mipsel-openwrt-linux-c++ mipsel-openwrt-linux-musl-as mipsel-openwrt-linux-musl-objcopy
mipsel-openwrt-linux-c++filt mipsel-openwrt-linux-musl-c++ mipsel-openwrt-linux-musl-objdump
mipsel-openwrt-linux-cpp mipsel-openwrt-linux-musl-c++filt mipsel-openwrt-linux-musl-ranlib
mipsel-openwrt-linux-elfedit mipsel-openwrt-linux-musl-cpp mipsel-openwrt-linux-musl-readelf
mipsel-openwrt-linux-g++ mipsel-openwrt-linux-musl-elfedit mipsel-openwrt-linux-musl-size
mipsel-openwrt-linux-gcc mipsel-openwrt-linux-musl-g++ mipsel-openwrt-linux-musl-strings
mipsel-openwrt-linux-gcc-8.4.0 mipsel-openwrt-linux-musl-gcc mipsel-openwrt-linux-musl-strip
mipsel-openwrt-linux-gcc-ar mipsel-openwrt-linux-musl-gcc-8.4.0 mipsel-openwrt-linux-nm
mipsel-openwrt-linux-gcc-nm mipsel-openwrt-linux-musl-gcc-ar mipsel-openwrt-linux-objcopy
mipsel-openwrt-linux-gcc-ranlib mipsel-openwrt-linux-musl-gcc-nm mipsel-openwrt-linux-objdump
mipsel-openwrt-linux-gcov mipsel-openwrt-linux-musl-gcc-ranlib mipsel-openwrt-linux-ranlib
mipsel-openwrt-linux-gcov-dump mipsel-openwrt-linux-musl-gcov mipsel-openwrt-linux-readelf
mipsel-openwrt-linux-gcov-tool mipsel-openwrt-linux-musl-gcov-dump mipsel-openwrt-linux-size
mipsel-openwrt-linux-gdb mipsel-openwrt-linux-musl-gcov-tool mipsel-openwrt-linux-strings
mipsel-openwrt-linux-gprof mipsel-openwrt-linux-musl-gdb mipsel-openwrt-linux-strip
mipsel-openwrt-linux-ld mipsel-openwrt-linux-musl-gprof
root@f9d79fd30362:/home/backup/crosscompile# mipsel-openwrt-linux-
```

Hình 2. 23: Kiểm tra toolchain.

- Tạo một file helloworld.c:

```
touch helloworld.c
nano helloworld.c
root@f9d79fd30362:/home/backup/crosscompile# touch helloworld.c
root@f9d79fd30362:/home/backup/crosscompile# ls
applications helloworld.c led_gpio11.c mqtt_ai7688 mqtt_ai7688_json staging_dir
root@f9d79fd30362:/home/backup/crosscompile# nano helloworld.c
```

Hình 2. 24: Tạo file helloworld.c

- Code helloworld.c:

```
#include<stdio.h>

int main()
{
    printf("Hello World, Lab AIoT\n");
    return 0;}
```

- Biên dịch file helloworld.c:

```
mipsel-openwrt-linux-musl-gcc helloworld.c -o helloworld
```

```

root@f9d79fd30362:/home/backup/crosscompile# ls
applications helloworld.c led_gpio11.c mqtt_ai7688 mqtt_ai7688_json staging_dir
root@f9d79fd30362:/home/backup/crosscompile# mipsel-openwrt-linux-musl-gcc helloworld.c -o helloworld
root@f9d79fd30362:/home/backup/crosscompile# ls
applications helloworld helloworld.c led_gpio11.c mqtt_ai7688 mqtt_ai7688_json staging_dir
root@f9d79fd30362:/home/backup/crosscompile#

```

Hình 2. 25: Biên dịch file helloworld.c

- Chuyển file helloworld sang Ai7688H:

```
scp helloworld root@192.168.1.132: /root
```

```
chmod 777 helloworld
```

```
./helloworld
```

```

root@OpenWrt:~# ls
helloworld
root@OpenWrt:~# chmod 777 helloworld
root@OpenWrt:~# ls
helloworld
root@OpenWrt:~# ./helloworld
Hello World, Lab AIoT
root@OpenWrt:~#

```

OpenWrt 3% 0,06 GB / 0,12 GB 0,05 Mb/s 0,01 Mb/s 178 min

Hình 2. 26: Chạy file helloworld vừa biên dịch xong ở board Ai7688.

2.2.6. Thêm Package

- Tại thư mục openwrt, tạo 1 folder helloworld:

```
mkdir helloworld
```

```

root@f9d79fd30362:/home/backup/openwrt# ls
BSDmakefile Makefile build_dir feeds.conf.default key-build.pub mqtt_ai7688 scripts toolchain
COPYING README.md config helloworld key-build.ucert mypackages staging_dir tools
Config.in bin dl include key-build.ucert.revoke package target version
LICENSES blink_led feeds key-build logs rules.mk tmp version.date
root@f9d79fd30362:/home/backup/openwrt#

```

Hình 2. 27: Tạo thư mục chứa file helloworld.c

- Trong thư mục helloworld tạo 1 file helloworld.c:

```
touch helloworld.c
```

```
nano helloworld.c
```

```

root@f9d79fd30362:/home/backup/openwrt# ls
BSDmakefile  Makefile  build_dir  feeds.conf.default  key-build.pub  mqtt_ai7688  scripts  toolchain
COPYING      README.md  config     helloworld         key-build.ucert  mypackages  staging_dir  tools
Config.in    bin       dl         include            key-build.ucert.revoke  package  target  version
LICENSES     blink_led feeds      key-build          logs            rules.mk  tmp      version.date
root@f9d79fd30362:/home/backup/openwrt# cd helloworld/
root@f9d79fd30362:/home/backup/openwrt/helloworld# ls
hello_world.cpp  helloworld.c
root@f9d79fd30362:/home/backup/openwrt/helloworld# █

```

Hình 2. 28: Tạo file `helloworld.c` để làm package

➤ Code `helloworld.c`:

```

#include<stdio.h>

int main()
{
    printf("Hello World, Lab AIoT!\n");

    return 0;
}

```

➤ Tại thư mục `openwrt`, tạo 1 file `mypackages`:

<code>mkdir mypackages</code>
<code>cd mypackages</code>
<code>mkdir example</code>
<code>cd example</code>
<code>mkdir helloworld</code>
<code>cd helloworld</code>
<code>touch Makefile</code>
<code>nano Makefile</code>

```

root@f9d79fd30362:/home/backup/openwrt# ls
BSDmakefile  Makefile  build_dir  feeds.conf.default  key-build.pub  mqtt_ai7688  scripts  toolchain
COPYING      README.md  config     helloworld         key-build.ucert  mypackages  staging_dir  tools
Config.in    bin       dl         include            key-build.ucert.revoke  package  target  version
LICENSES     blink_led feeds      key-build          logs            rules.mk  tmp      version.date
root@f9d79fd30362:/home/backup/openwrt# cd mypackages/examples/helloworld/
root@f9d79fd30362:/home/backup/openwrt/mypackages/examples/helloworld# ls
Makefile
root@f9d79fd30362:/home/backup/openwrt/mypackages/examples/helloworld# █

```

Hình 2. 29: Tạo file Makefile cho package helloworld

➤ Code Makefile:

```
include $(TOPDIR)/rules.mk

# Name, version and release number
# The name and version of your package are used to define the variable to point
to the build directory of your package: $(PKG_BUILD_DIR)
PKG_NAME:=helloworld
PKG_VERSION:=1.0
PKG_RELEASE:=1

# Source settings (i.e. where to find the source codes)
# This is a custom variable, used below, duong dan vao file helloworld.c
SOURCE_DIR:=/home/backup/openwrt/helloworld
include $(INCLUDE_DIR)/package.mk

# Package definition; instructs on how and where our package will appear in the
overall configuration menu ('make menuconfig')
define Package/helloworld
    SECTION:=examples
    CATEGORY:=Examples
    TITLE:=Hello, World!
endef

# Package description; a more verbose description on what our package does
define Package/helloworld/description
    A simple "Hello, world!" -application.
endef
```

```
# Package preparation instructions; create the build directory and copy the source
code.

# The last command is necessary to ensure our preparation instructions remain
compatible with the patching system.

define Build/Prepare
    mkdir -p $(PKG_BUILD_DIR)
    cp $(SOURCE_DIR)/* $(PKG_BUILD_DIR)
    $(Build/Patch)
endef

# Package build instructions; invoke the target-specific compiler to first compile
the source file, and then to link the file into the final executable

define Build/Compile
    $(TARGET_CC) $(TARGET_CFLAGS) -o
$(PKG_BUILD_DIR)/helloworld.o -c $(PKG_BUILD_DIR)/helloworld.c
    $(TARGET_CC) $(TARGET_LDFLAGS) -o $(PKG_BUILD_DIR)/$1
$(PKG_BUILD_DIR)/helloworld.o
endef

# Package install instructions; create a directory inside the package to hold our
executable, and then copy the executable we built previously into the folder

define Package/helloworld/install
    $(INSTALL_DIR) $(1)/usr/bin
    $(INSTALL_BIN) $(PKG_BUILD_DIR)/helloworld $(1)/usr/bin
endef

# This command is always the last, it uses the definitions and variables we give
above in order to get the job done

$(eval $(call BuildPackage,helloworld))
```

➤ Tại thư mục openwrt, cập nhật lại package:

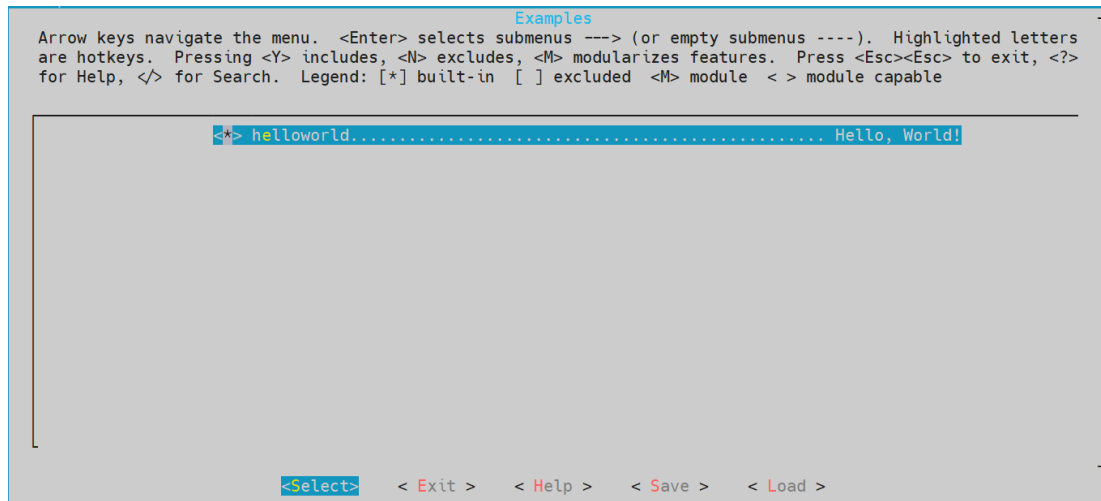
```
./scripts/feeds update -a
```

```
./scripts/feeds install -a
```

```
root@96b961809d6f:/home/backup/openwrt/helloworld# ls
helloworld.c
root@96b961809d6f:/home/backup/openwrt/helloworld# cd ..
root@96b961809d6f:/home/backup/openwrt# ls
BSDmakefile  Makefile  build_dir  feeds.conf.default  key-build.pub  mqtt_ai7688  scripts  toolchain
COPYING      README.md  config     helloworld          key-build.ucert  mypackages  staging_dir  tools
Config.in    bin       dl         include             key-build.ucert.revoke  package     target      version
LICENSES     blink_led feeds      key-build           logs            rules.mk    tmp         version.date
root@96b961809d6f:/home/backup/openwrt# cd mypackages/examples/helloworld/
root@96b961809d6f:/home/backup/openwrt/mypackages/examples/helloworld# ls
Makefile
root@96b961809d6f:/home/backup/openwrt/mypackages/examples/helloworld# cd ../../..
root@96b961809d6f:/home/backup/openwrt# ./scripts/feeds update -a
Updating feed 'packages' from 'https://git.openwrt.org/feed/packages.git^78bcd00c13587571b5c79ed2fc3363aa674aaef7' ...
Create index file './feeds/packages.index'
Updating feed 'luci' from 'https://git.openwrt.org/project/luci.git^fe09ab990256d8b63e76cc1ab2435baef921b8b4' ...
Create index file './feeds/luci.index'
Updating feed 'routing' from 'https://git.openwrt.org/feed/routing.git^a0d61bddb3ce4ca54bd76af86c28f58feb6cc044' ...
Create index file './feeds/routing.index'
Updating feed 'telephony' from 'https://git.openwrt.org/feed/telephony.git^0183c1adda0e7581698b0ea4bff7c08379acf447' ...
Create index file './feeds/telephony.index'
Updating feed 'mypackages' from '/home/backup/openwrt/mypackages' ...
Create index file './feeds/mypackages.index'
root@96b961809d6f:/home/backup/openwrt# ./scripts/feeds install -a
Installing all packages from feed packages.
Installing all packages from feed luci.
Installing all packages from feed routing.
Installing all packages from feed telephony.
Installing all packages from feed mypackages.
root@96b961809d6f:/home/backup/openwrt#
```

Hình 2. 30: Cập nhật lại Package.

- Mở Menuconfig và tích gói helloworld:

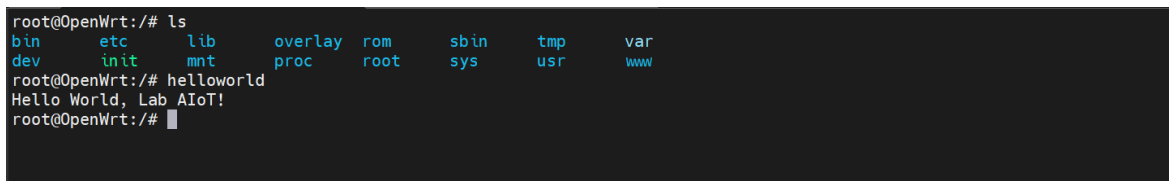


Hình 2. 31: Chọn gói helloworld trên menuconfig.

- Build lại image:

```
make -j24 V=99
```

- Nạp lại image vừa build cho AI7688H và ta được kết quả như sau:



Hình 2. 32: Chạy thử package helloworld vừa thêm.

2.2.7. Cấu hình Device Tree

- Sửa file mt7628an.dtsi:

```
uart0_pins: uartlite {  
    uartlite {  
        groups = "uart0";  
        function = "gpio";  
    };  
};
```

-
- Cấu hình thêm GPIO tại file `mt7628an_mediatek_linkit-smart-7688.dts`:

```
leds {
    compatible = "gpio-leds";

    led_wifi: wifi {
        label = "orange:wifi";
        gpios = <&gpio 44 GPIO_ACTIVE_LOW>;
    };

    led_12: Tx {
        label = "led_yellow";
        gpios = <&gpio 12 GPIO_ACTIVE_HIGH>;
    };

    led_13: Rx {
        label = "led_red";
        gpios = <&gpio 13 GPIO_ACTIVE_LOW>;
    };
};
```

- Build lại image và nạp lại image cho AI7688:

```
make -j24 V=99
```

2.3. Thử nghiệm

- Code file `mqtt_pub.cpp`:

```
#include <mosquitto.h>
#include <mqtt_protocol.h>
```

```
#include <jsoncpp/json/json.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <iostream>

#define HOST    "Mqtt.mysignage.vn"
#define PORT    1883
#define PUB_TOPIC  "BLINK_LED"
#define CLIENT_ID  "client_pub"

using namespace std;

int count = 0, flag = 0;
const char* msg_led;

struct mosquitto *mosq;

int main(int argc, char * argv[])
{
    mosquitto_lib_init();
    mosq = mosquitto_new(CLIENT_ID, true, NULL);

    if(mosq)
    {
        mosquitto_connect(mosq, HOST, PORT, 60);
    }

    while(1)
    {
```

```
    Json::Value message;
    message["Device"] = "Led";

    if(flag == 1)
    {
        flag = 0;
        message["Value_led"] = 1;
    }
    else
    {
        flag = 1;
        message["Value_led"] = 0;
    }

    string json_string;
    Json::FastWriter writer;
    json_string = writer.write(message);
    msg_led = json_string.c_str();

    cout << "Connection active!" << endl;
    cout << "Number message: " << count++ << endl;
    cout << msg_led << endl;

    mosquitto_publish(mosq, NULL, PUB_TOPIC, strlen(msg_led), msg_led,
0, false);
    sleep(2);

    mosquitto_loop(mosq, -1, 1);
}
}
```

➤ Code file mqtt_sub.cpp:

```
#include <mosquitto.h>
#include <mqtt_protocol.h>
#include <json/json.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <iostream>
#include <fstream>

#define HOST    "Mqtt.mysignage.vn"
#define PORT    1883
#define SUB_TOPIC "BLINK_LED"
#define CLIENT_ID "client_sub"

using namespace std;

struct mosquitto *mosq;
string message_topic_str;
string message_payload_str;
std::ofstream outputFile("/sys/class/leds/led_red/brightness", std::ios::trunc);

void message_callback(struct mosquitto *mosq, void *obj, const struct
mosquitto_message *message)
{
    cout << "Got message from " << message->topic << " topic" << endl;
    cout << "Content: " << (char*) message->payload << endl;
    message_payload_str = (char*) message->payload;
```

```
message_topic_str = message->topic;
Json::Reader reader;
Json::Value root;
Json::FastWriter writer;

if (reader.parse(message_payload_str, root))
{
    if (root.isMember("Value_led") && root["Value_led"].isInt())
    {
        int value_led = root["Value_led"].asInt();
        cout << "Value_led: " << value_led << endl;
        if(value_led == 1)
        {
            outputFile << 1;
            outputFile.flush();
        }
        else
        {
            outputFile << 0;
            outputFile.flush();
        }
    }
    else
    {
        cout << "Invalid or missing Value_led field in the JSON message." <<
endl;
    }
}
else
{

```

```
        cout << "Failed to parse the JSON message." << endl;
    }
}

int main(int argc, char * argv[])
{
    if (!outputFile.is_open())
    {
        cerr << "Failed to open the /sys/class/leds/led_red/brightness file." << endl;
        return 1;
    }

    mosquitto_lib_init();
    mosq = mosquitto_new(CLIENT_ID, true, NULL);

    if(mosq)
    {
        mosquitto_connect(mosq, HOST, PORT, 60);
        mosquitto_message_callback_set(mosq, message_callback);
        mosquitto_subscribe(mosq, NULL, SUB_TOPIC, 0);
    }

    while(1)
    {
        mosquitto_loop(mosq, 3000, 1);
    }
}
```

➤ Cross compile 2 file mqtt_pub.cpp và mqtt_sub.cpp:

```
mipsel-openwrt-linux-musl-g++ mqtt_sub.cpp -o mqtt_sub -lmosquitto -ljsoncpp
```



Hình 2. 33: Gửi 1 topic publish để blinkled trên board AI7688.

```
-----
root@OpenWrt:~# ./mqtt_sub
Got message from BLINK_LED topic
Content: {"Device":"Led","Value_led":0}

Got message from BLINK_LED topic
Content: {"Device":"Led","Value_led":1}

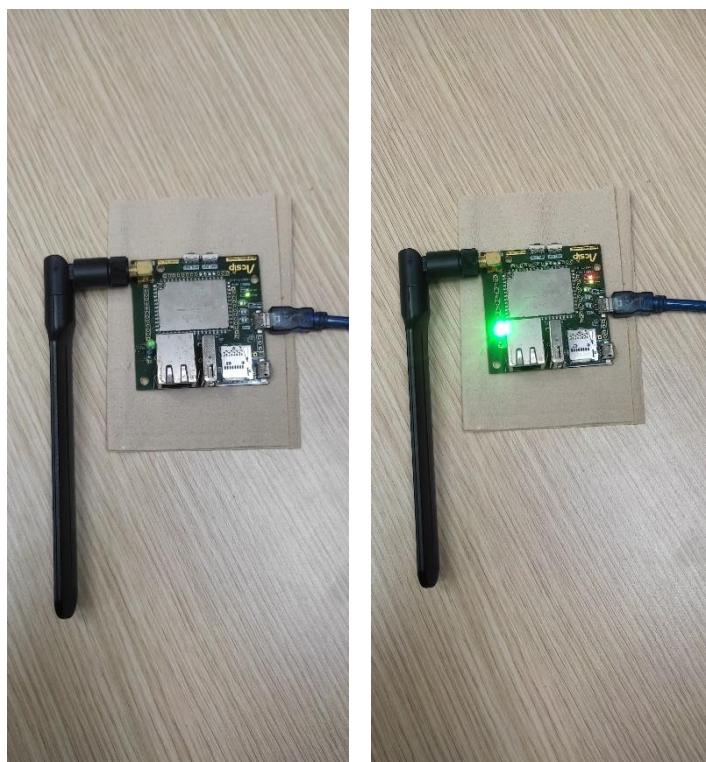
Got message from BLINK_LED topic
Content: {"Device":"Led","Value_led":0}

Got message from BLINK_LED topic
Content: {"Device":"Led","Value_led":1}

Got message from BLINK_LED topic
Content: {"Device":"Led","Value_led":0}

-----
OpenWrt 17% 0,06 GB / 0,12 GB 0,07 Mb/s 0,02 Mb/s 187 min
```

Hình 2. 34: Board AI7688 nhận được bản tin topic publish.



Hình 2. 35: Thiết bị chạy thật.

KẾT LUẬN

Sau quá trình thực tập tại phòng lab Rạng Đông – Học viện công nghệ bưu chính viễn thông, em đã học hỏi được rất nhiều kiến thức từ cơ bản đến chuyên sâu, từ lý thuyết đến thực tế. Em đã được tham gia vào vị trí nghiên cứu các thuật toán phá hiện bất thường tại biên, ngoài ra em còn được thực hành tạo một bản hệ điều hành linux cho các thiết bị nhúng... qua đó giúp em củng cố thêm các kiến thức đã học trên trường cũng như có thêm kinh nghiệm thực tế với các thiết bị.

Em xin gửi lời cảm ơn sâu sắc đến các thầy cô trong Khoa Điện tử I đã tạo điều kiện cho em được tham gia thực tập tốt nghiệp, cảm ơn ban lãnh đạo khoa và học viện, đặc biệt là thầy Nguyễn Ngọc Minh đã nhiệt tình giúp đỡ em trong thời gian thực tập.

Em xin chân thành cảm ơn!

TÀI LIỆU THAM KHẢO

- [1] Colby R Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, et al. 2020. Benchmarking TinyML Systems: Challenges and Direction. In SysML 2020, Proceedings. York.
- [2] Abbas Nasir, Zhang Yan, Taherkordi Amir, and Skeie Tor. 2017. Mobile edge computing: A survey. IEEE IoT J. 5, 1 (2017), 450–465.
- [3] Visal Rajapakse, Ishan Karunanayake, Nadeem Ahmed, Intelligence at the Extreme Edge: A Survey on Reformable TinyML
- [4] <https://openwrt.org/docs/guide-developer/defining-firmware-partitions>
- [5] <https://openwrt.org/docs/guide-developer/build-image-with-3g-dongle-support>
- [6] https://elinux.org/Device_Tree_Reference